

بحث الكور 132

البحث المخصص للكور 132 عن الفورم بلدر

كريم رفعت

2016/02/17

بحث الكور 132

البحث المخصص للكور 132 عن الفورم بلدر

المحتويات

Index	Page
Form Builder	3
Creating Master Detial form	15
Working with data blocks and frames	23
Creating additional input items	36
Creating Noneinput items	46
Triggers	58
Navigation	62
Transaction Processing	67
Writing Flexible code	75
Sharing Objects and code	84

Form Builder

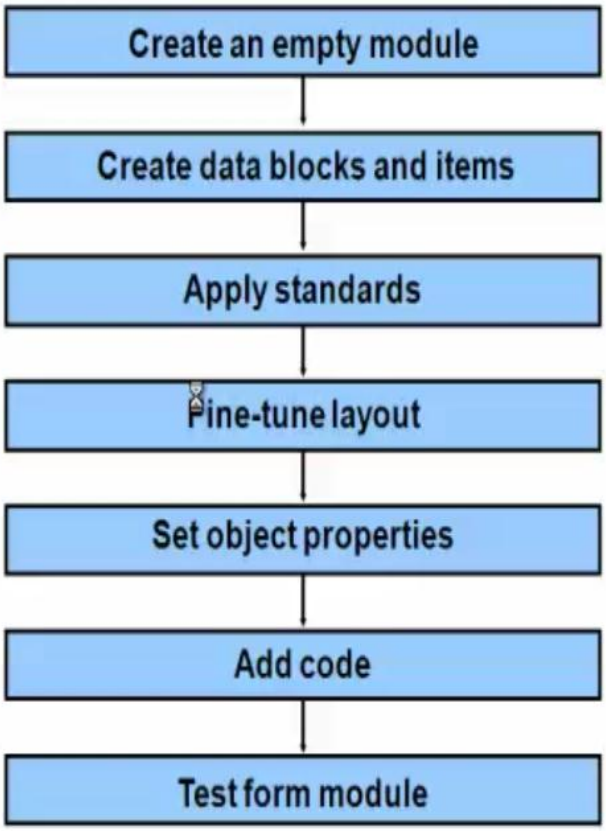
1 Creating a Basic Form Module

Objectives

After completing this lesson, you should be able to do the following:

- Create a form module
- Create a data block
- Save and compile a form module
- Identify Forms file formats and their characteristics
- Describe how to deploy a form module
- Explain how to create documentation for a Forms application

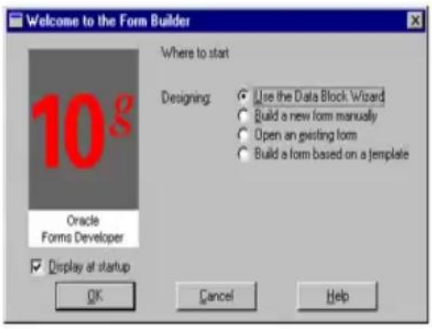
Creating a New Form Module



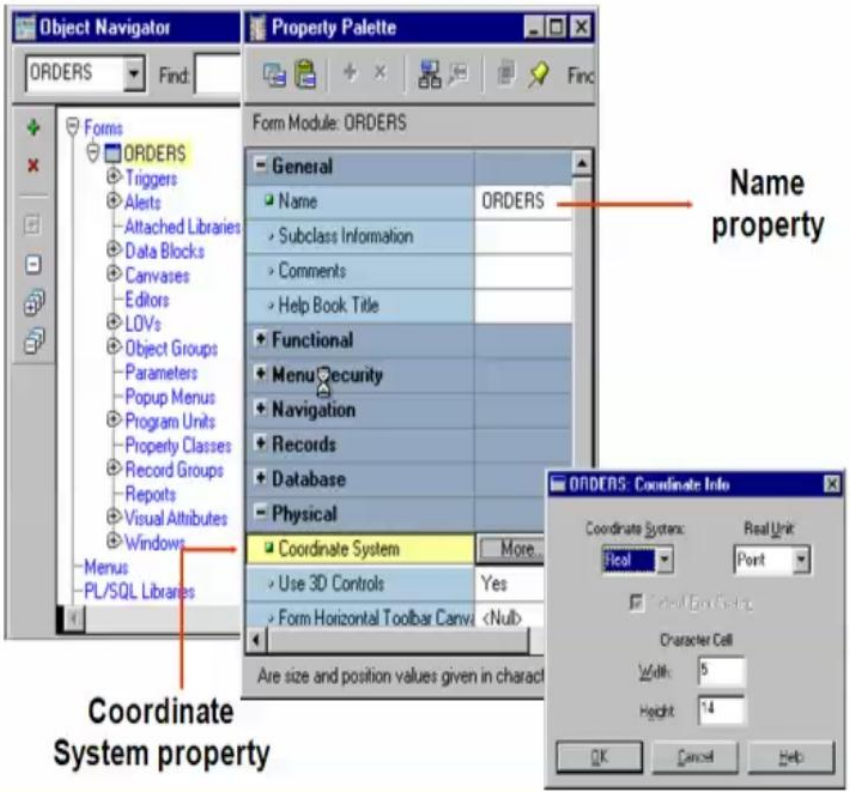
Creating a New Form Module

Choose one of the following methods:

- Use wizards:
  - Data Block Wizard
  - Layout Wizard
- Build module manually
- Use template form



Form Module Properties

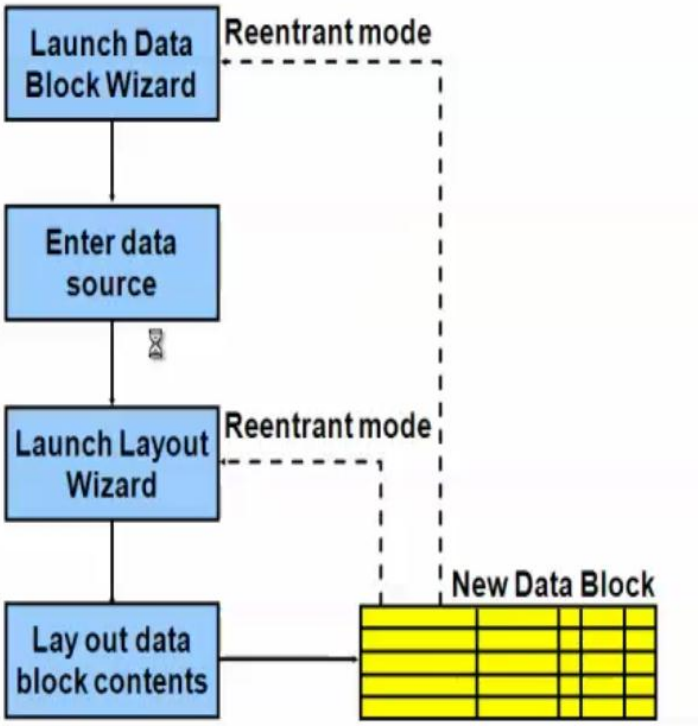


Creating a New Data Block

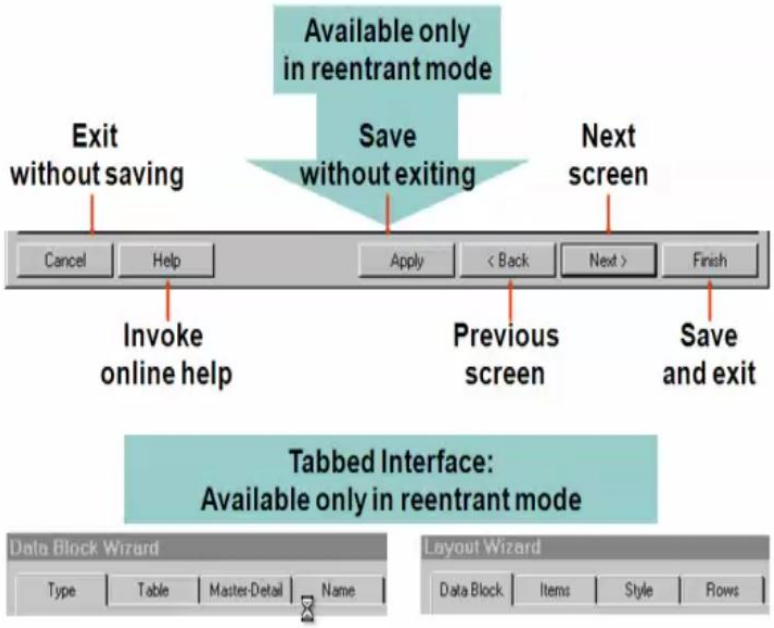
- **Use Forms Builder Wizards:**
  - Data Block Wizard: Create a data block with associated data source quickly and easily
  - Layout Wizard: Lay out data block contents for visual presentation
- **Create manually**



Creating a New Data Block



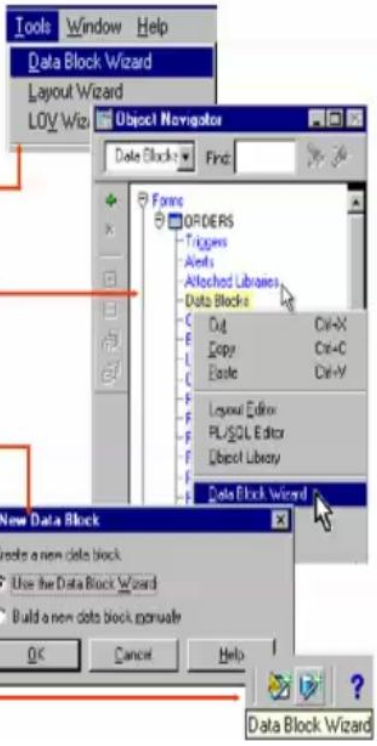
Navigating the Wizards



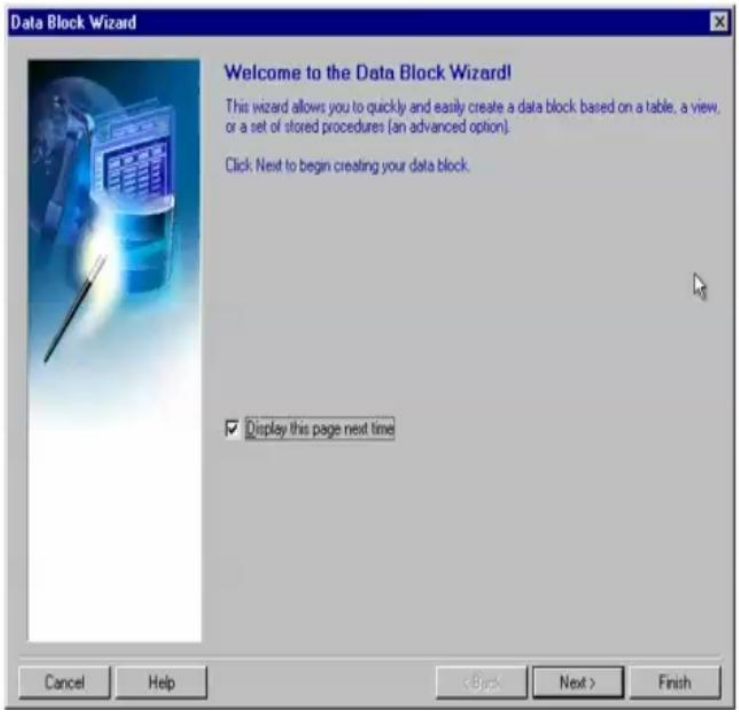
Launching the Data Block Wizard

In Forms Builder, do one of the following:

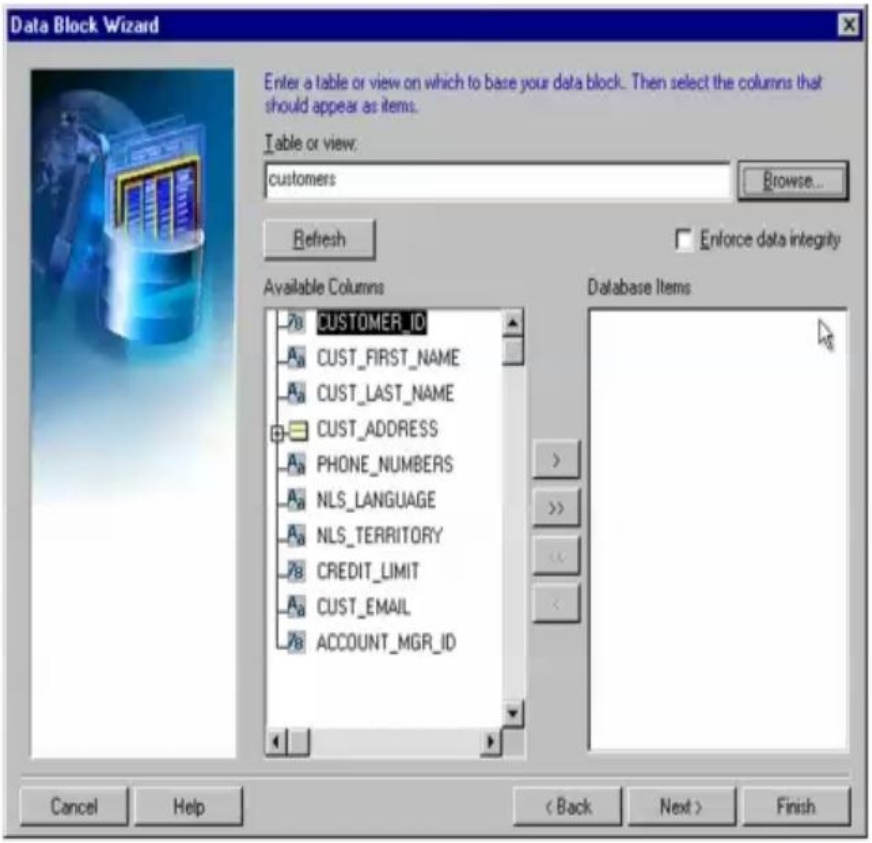
- Select Tools > Data Block Wizard.
- Right-click and select Data Block Wizard.
- Select the Data Blocks node and click Create icon; select Use the Data Block Wizard option.
- Use the Data Block Wizard button on the toolbar in the Layout Editor.



Data Block Wizard: Type Page



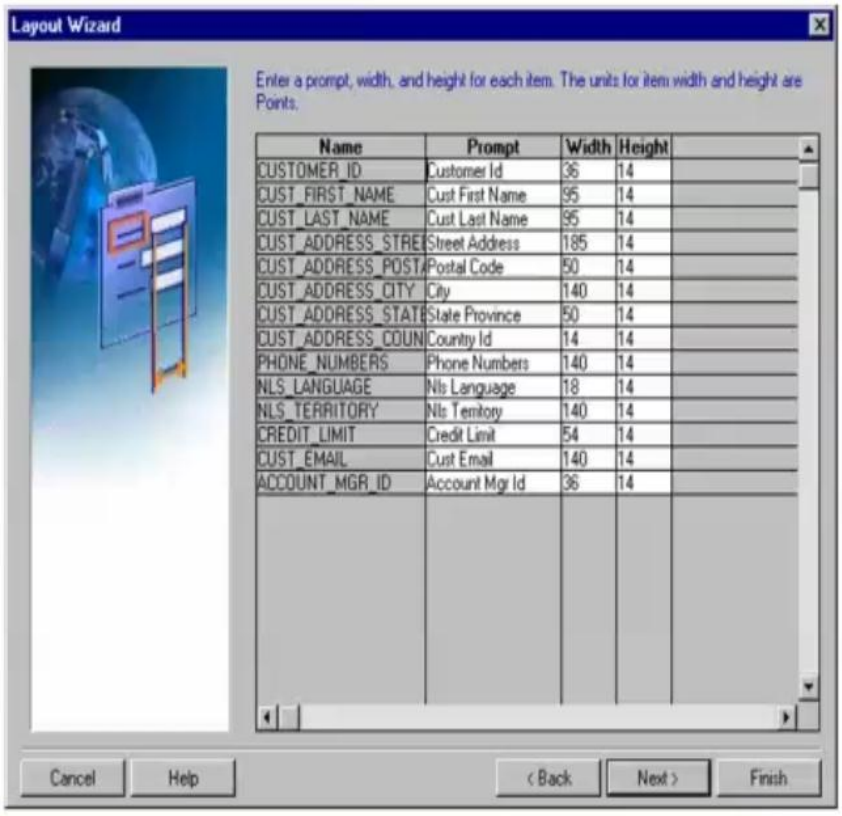
Data Block Wizard: Table Page



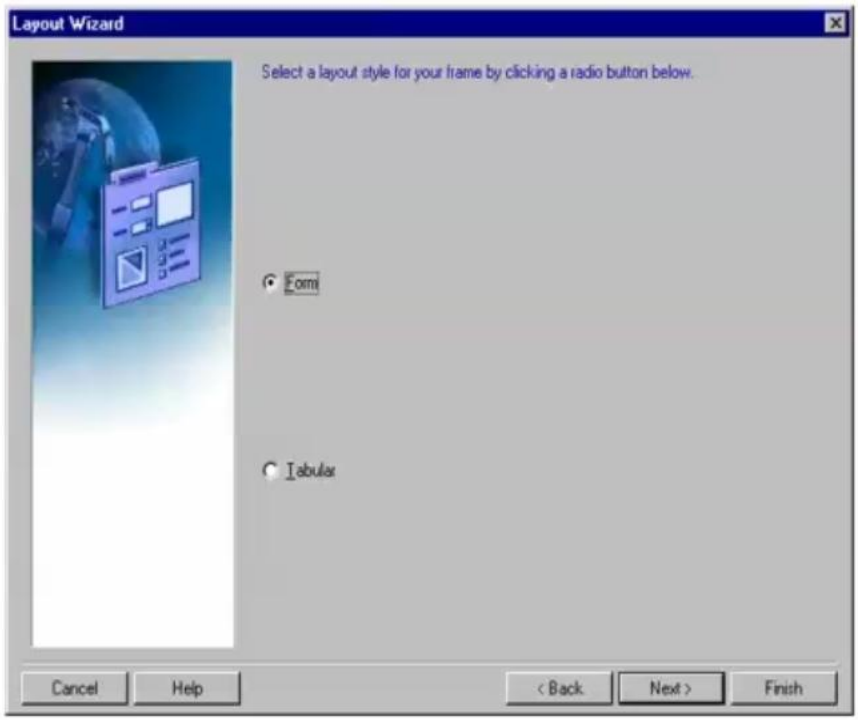
Data Block Wizard: Finish Page



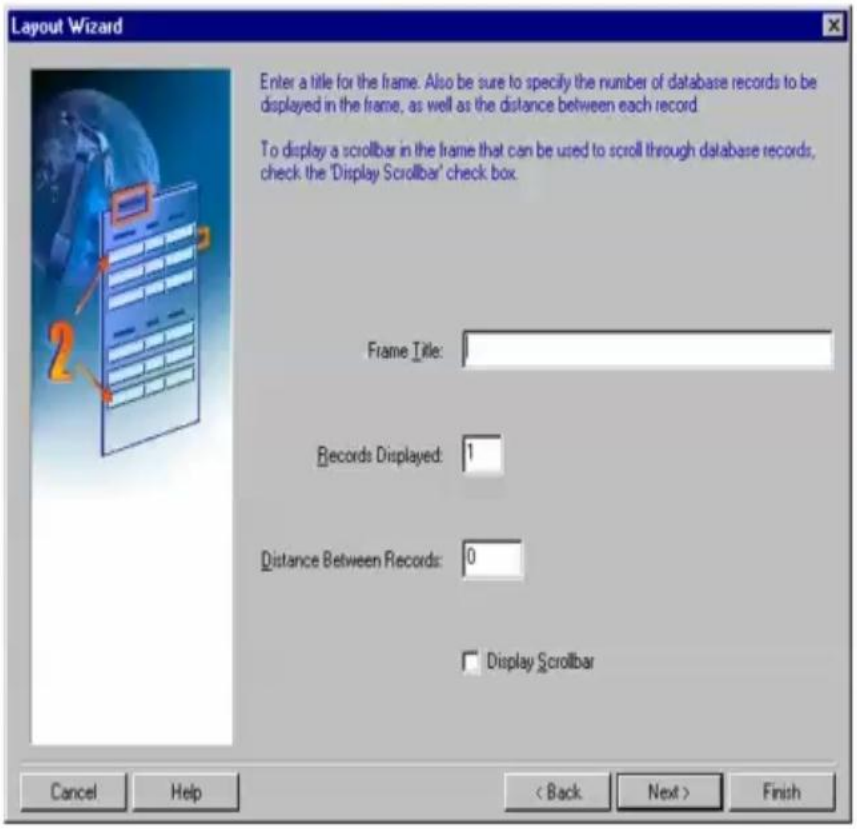
Layout Wizard: Items Page



Layout Wizard: Style Page



Layout Wizard: Rows Page

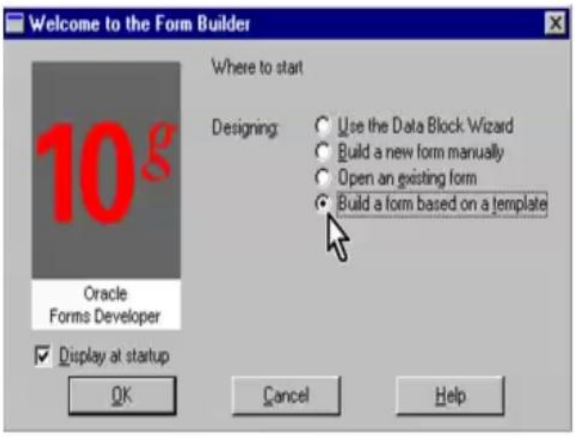


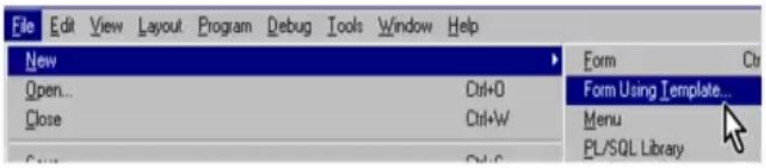
Data Block Functionality

Once you create a data block with the wizards, Forms Builder automatically creates:

- A form module with database functionality including query, insert, update, delete
- A frame object
- Items in the data block
- A prompt for each item
- Triggers needed to enforce database constraints if "Enforce data integrity" is checked

Template Forms





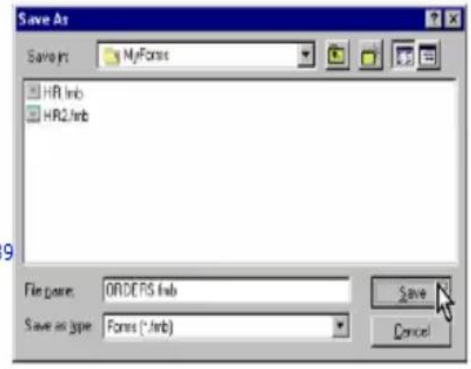
Saving a Form Module

To save the form module:

- Select File > Save
  OR
  Click the Save icon
- Enter a filename
- Navigate to desired location
- Click Save

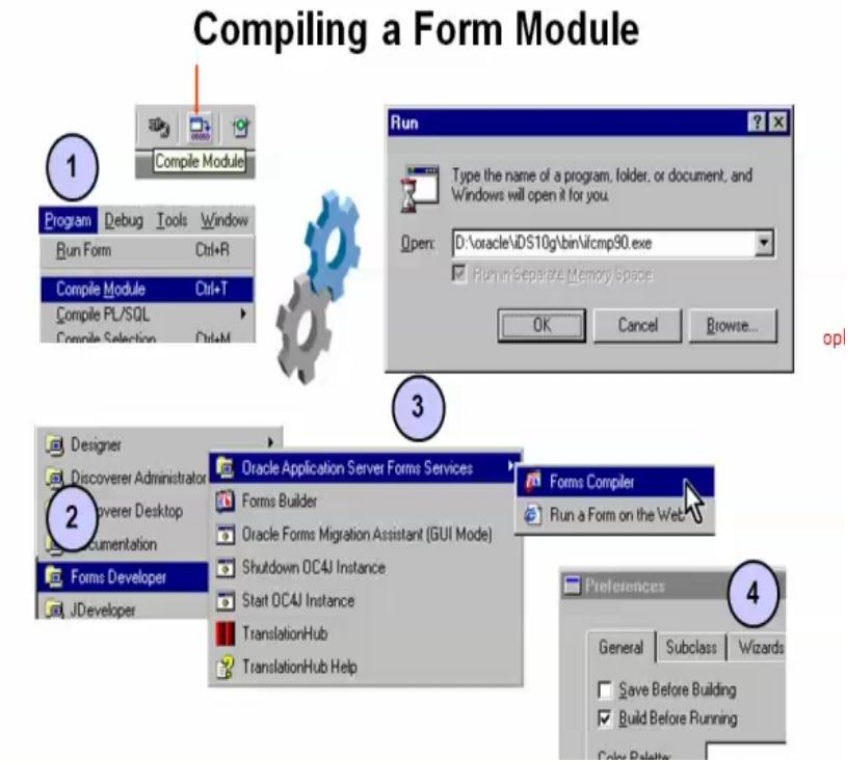






opkarim116939

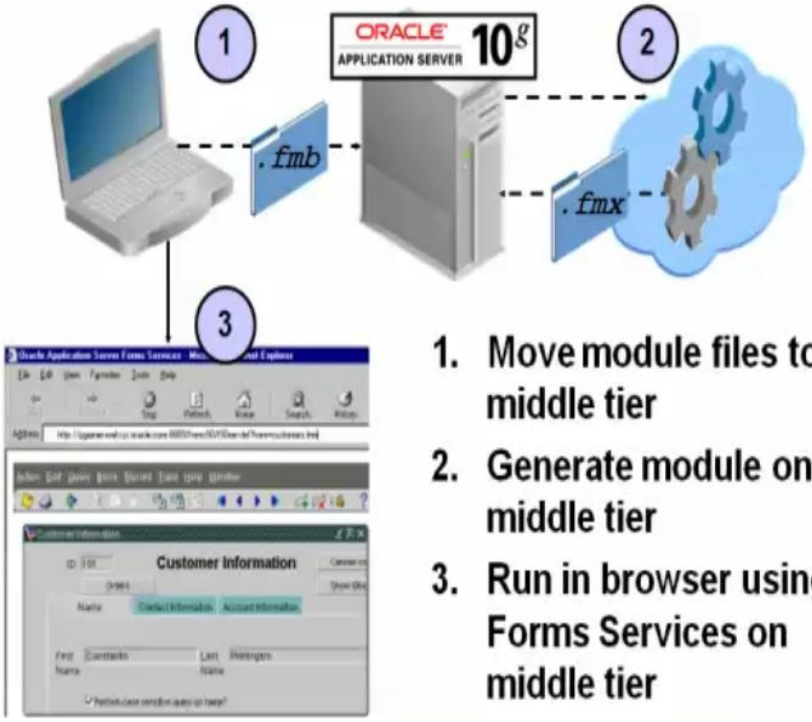
Compiling a Form Module



Module Types and Storage Formats

Form Module	.fmb	.fmx	.fmt
Menu Module	.mmb	.mmx	.mmt
PL/SQL Library	.pll	.plx	.pld
Object Library	.olb		.olt

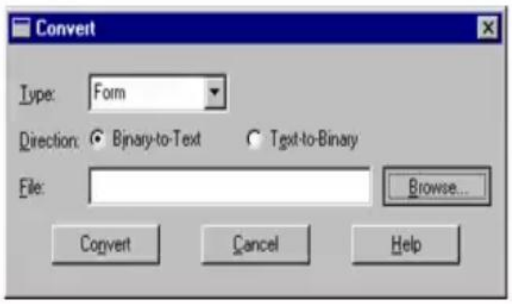
Deploying a Form Module



1. Move module files to middle tier
2. Generate module on middle tier
3. Run in browser using Forms Services on middle tier

Text Files and Documentation





- Convert a binary file to a text file.
- Create an ASCII file for a form module.

Summary

In this lesson, you should have learned that:

- To create a form module, you create an empty module, then add data blocks and other elements
- You can create a data block manually or with the Data Block Wizard and Layout Wizard
- You can save and compile a form module using the File and Program menus or from the toolbar
- You can store form, menu, and library modules in text format (useful for documentation), in a portable binary format, or a non-portable binary executable format
- To deploy a form module, you move it to the application server machine and generate it

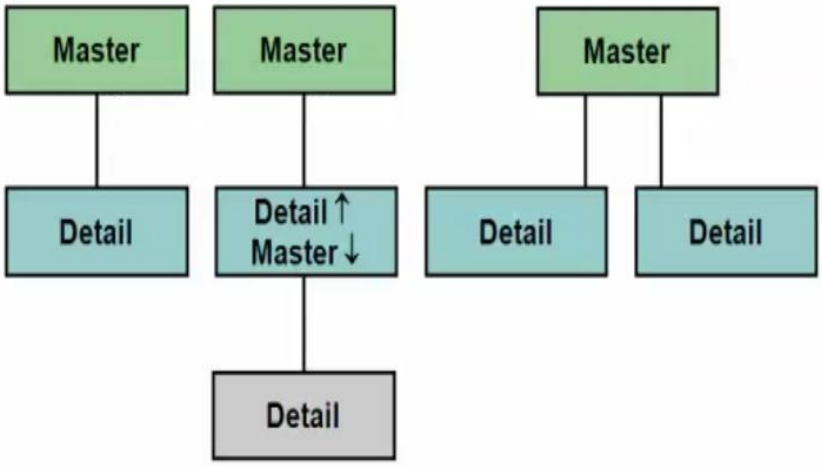
Creating a Master-Detail Form

Objectives

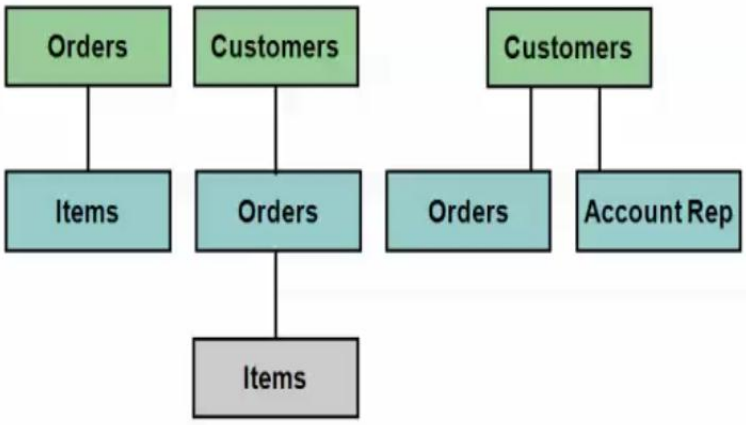
After completing this lesson, you should be able to do the following:

- Create data blocks with relationships
- Modify a data block
- Modify the layout of a data block
- Run a master-detail form

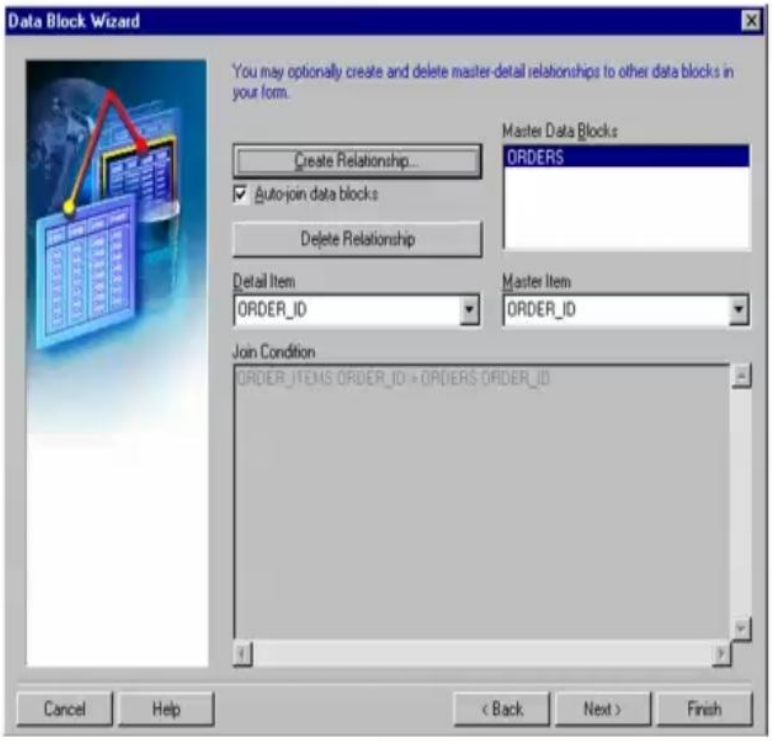
Form Block Relationships



Form Block Relationships



Data Block Wizard: Master-Detail Page



Relation Object

- New relation object created in Object Navigator under master data block node
- Default name assigned: `MasterDataBlock_DetailDataBlock`
- Triggers and program units generated automatically



Creating a Relation Manually



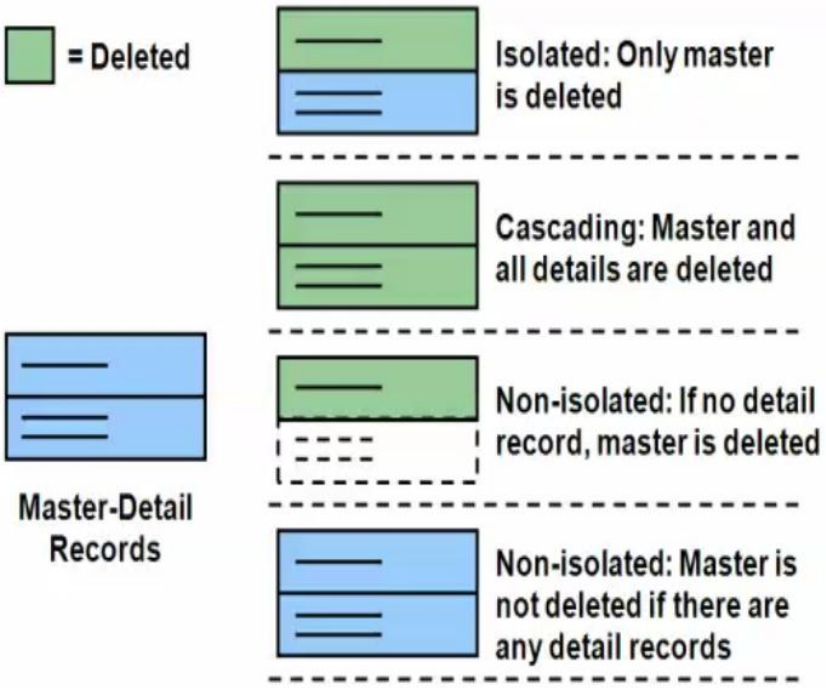


Join Condition

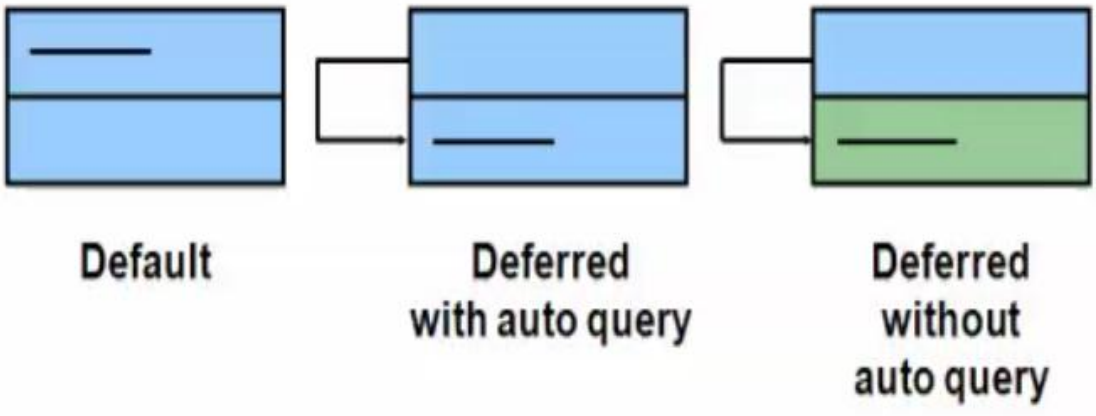
- The join condition creates primary-foreign key link between blocks
- Define a join condition using:
  - Block and item names (not table and column names)
  - Do not precede names with colon
  - SQL equijoin syntax



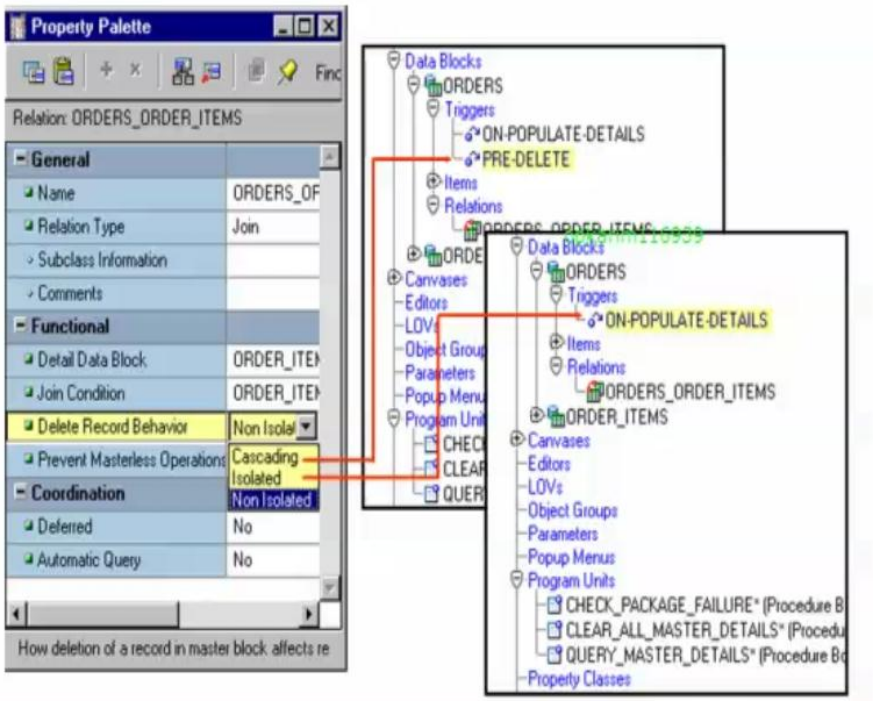
Deletion Properties



Coordination Properties

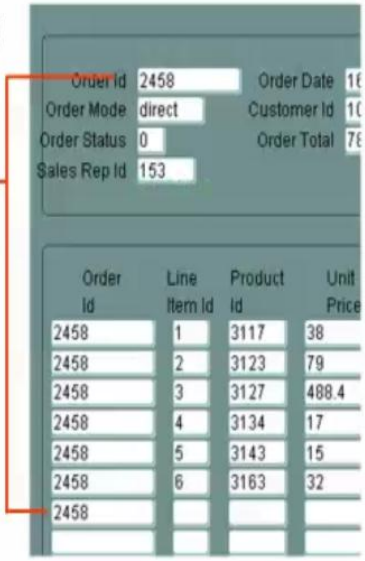


Modifying a Relation



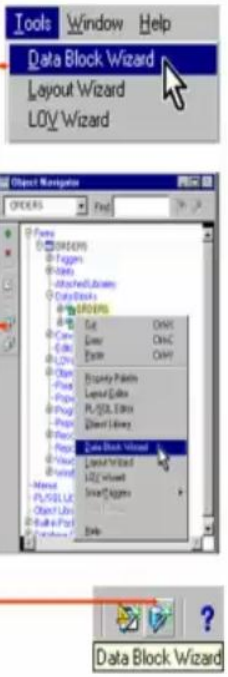
Running a Master-Detail Form Module

- **Automatic block linking for:**
  - Querying
  - Inserting
- **Default deletion rules: Cannot delete master record if detail records exist**



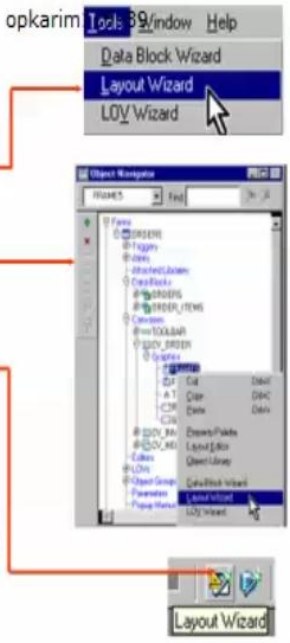
Modifying the Structure of a Data Block

- **Reentrant Data Block Wizard:**
  1. Select frame or object in Layout Editor, or data block or frame in Object Navigator
  2. Select Tools > Data Block Wizard OR
     Right-click and select Data Block Wizard OR
     Click Data Block Wizard
- **Object Navigator:**
  - Create or delete items
  - Change item properties
- **Block Property Palette: Change property values**



Modifying the Layout of a Data Block

- **Reentrant Layout Wizard:**
  - Select frame in Object Navigator or Layout Editor
  - Select Tools > Layout Wizard
  - OR
  - Right-click and select Layout Wizard OR
  - Click Layout Wizard
- **Layout Editor:**
  - Select Tools > Layout Editor
  - Make changes manually
- **Frame Property Palette: Change property values**



Summary

In this lesson, you should have learned that:

- You can create data blocks with relationships by using the Data Block Wizard or by manually creating a Relation object
- When you run a master-detail form, block coordination is automatic depending on properties of the Relation object
- You can modify a data block manually or with the Data Block Wizard in reentrant mode
- You can modify the layout manually or with the Layout Wizard in reentrant mode

Working with Data Blocks and Frames

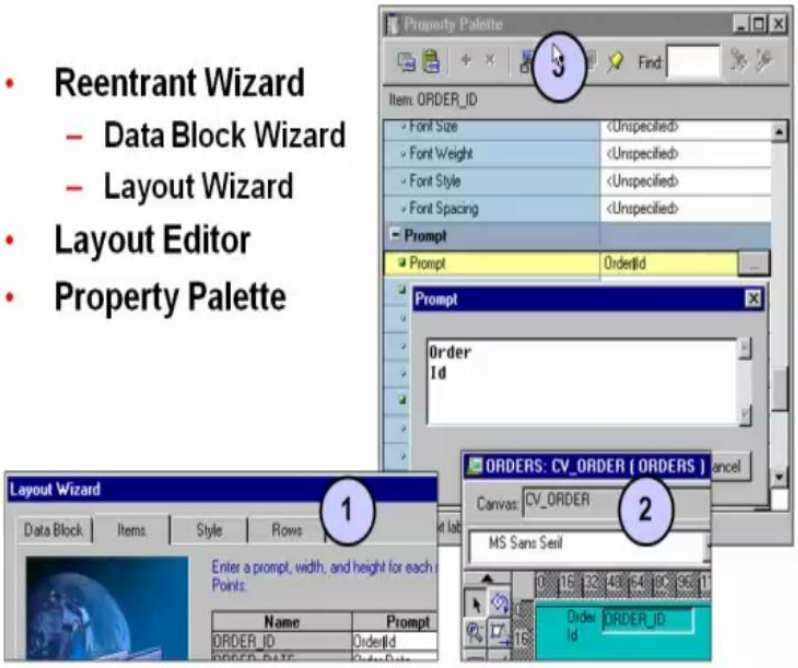
Objectives

After completing this lesson, you should be able to do the following:

- Identify the components of the Property Palette
- Manage object properties
- Create and use Visual Attributes
- Control the behavior and appearance of data blocks
- Control frame properties
- Create blocks that do not directly correspond to database tables
- Delete data blocks and their components

Managing Object Properties

- Reentrant Wizard
  - Data Block Wizard
  - Layout Wizard
- Layout Editor
- Property Palette

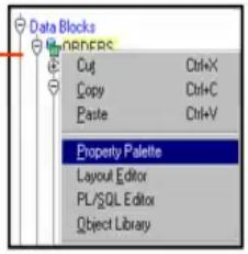


Displaying the Property Palette

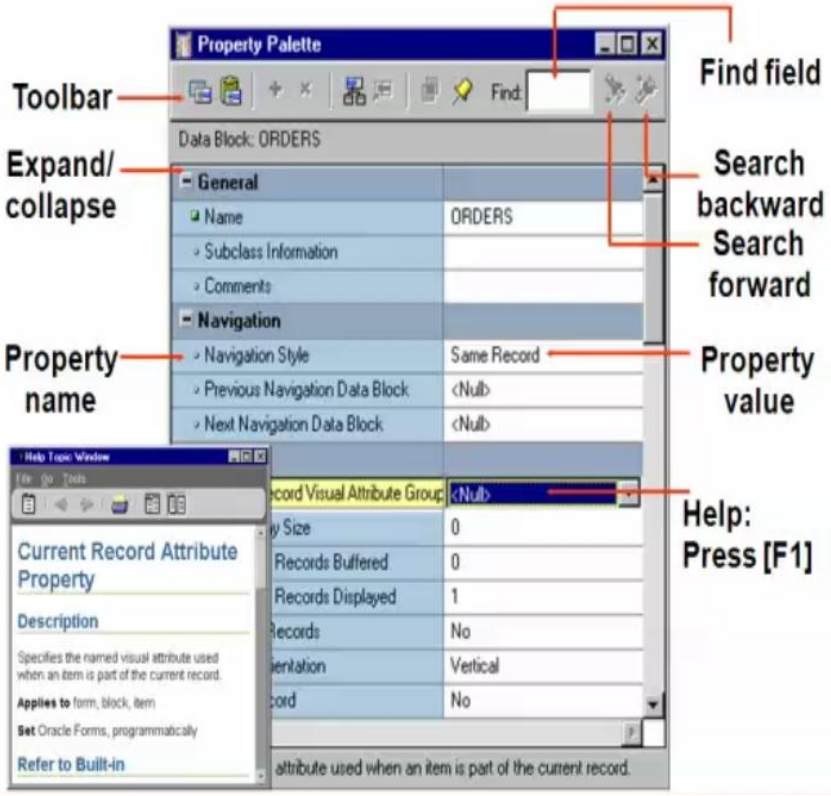


To display the Property Palette, use one of the following methods:

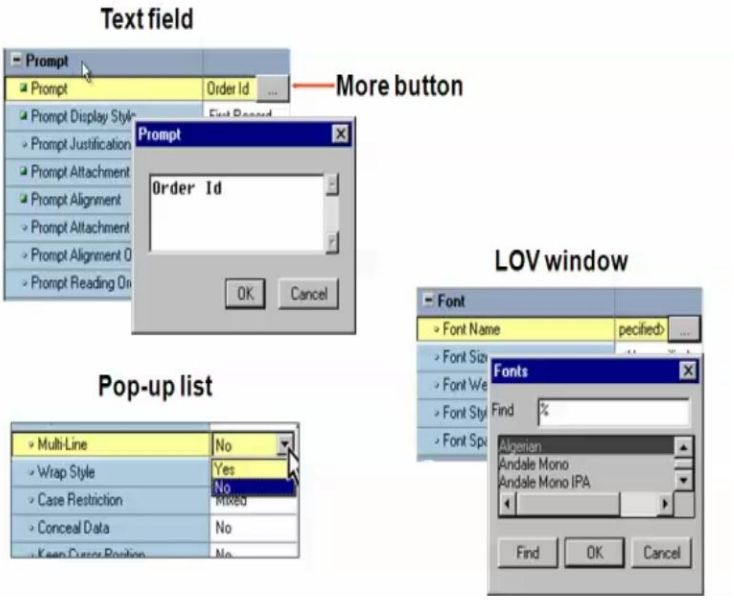
- Select Tools > Property Palette (or use the shortcut key).
- Double-click the object icon in the Object Navigator.
- Double-click the object in the Layout Editor.
- Right-click the object icon in the Object Navigator.
- Right-click the object in the Layout Editor.



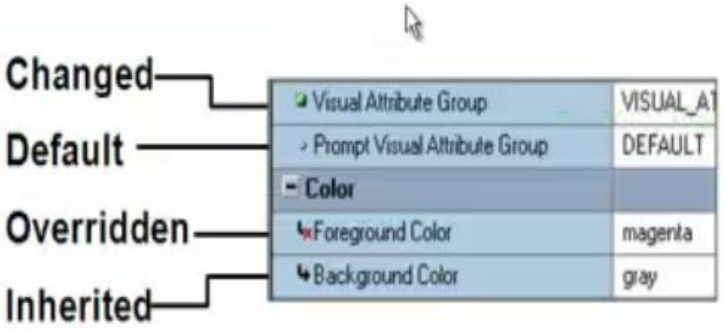
Property Palette: Features



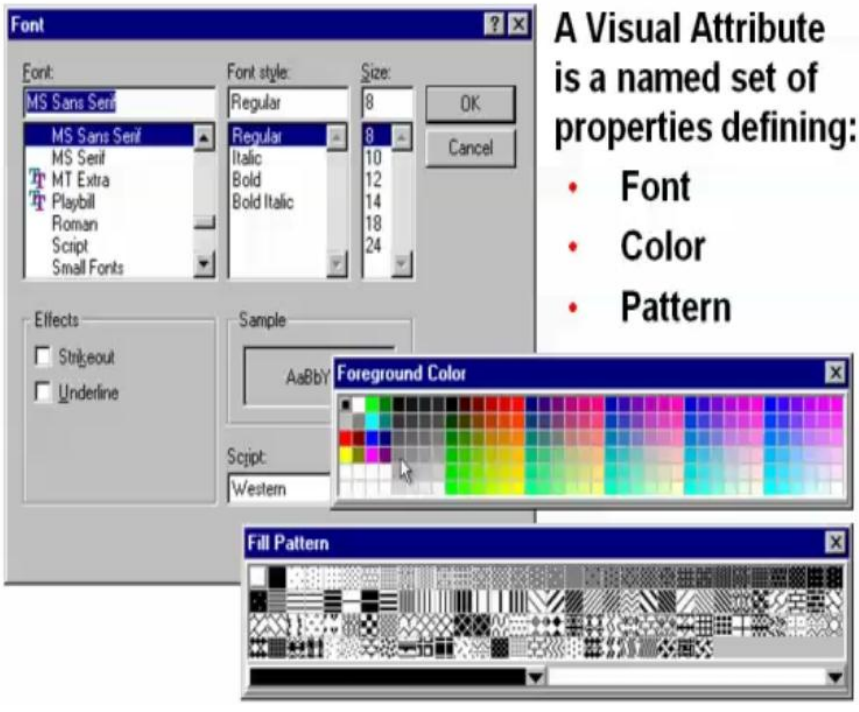
Property Controls



Property Controls



Visual Attributes

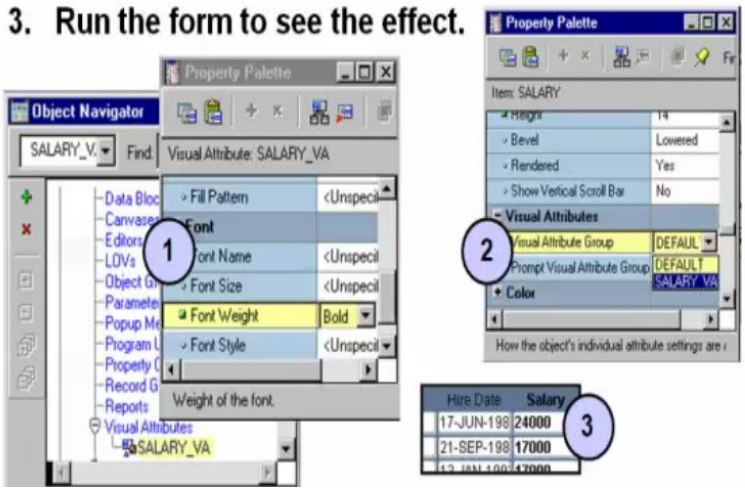


A Visual Attribute is a named set of properties defining:

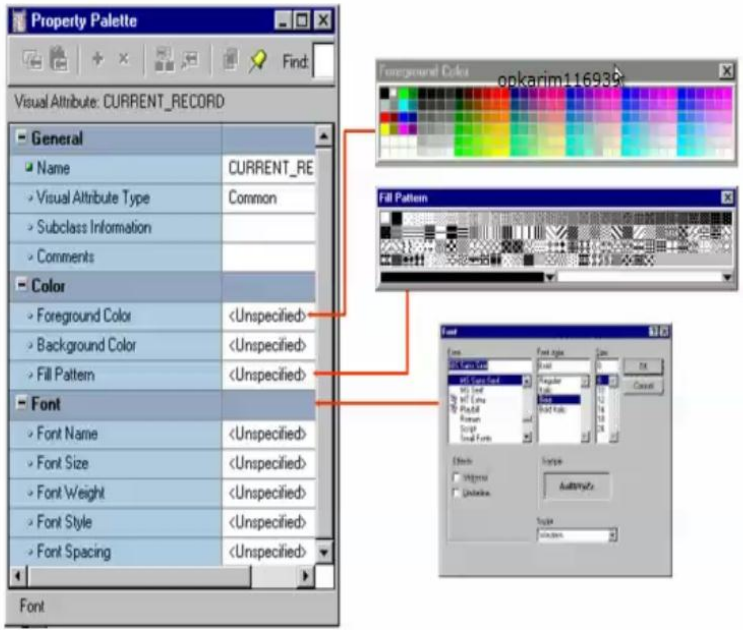
- Font
- Color
- Pattern

How to Use Visual Attributes

1. Create a Visual Attribute.
2. Set the Visual Attribute–related property of an object to the desired Visual Attribute.
3. Run the form to see the effect.



Font, Pattern, and Color Pickers



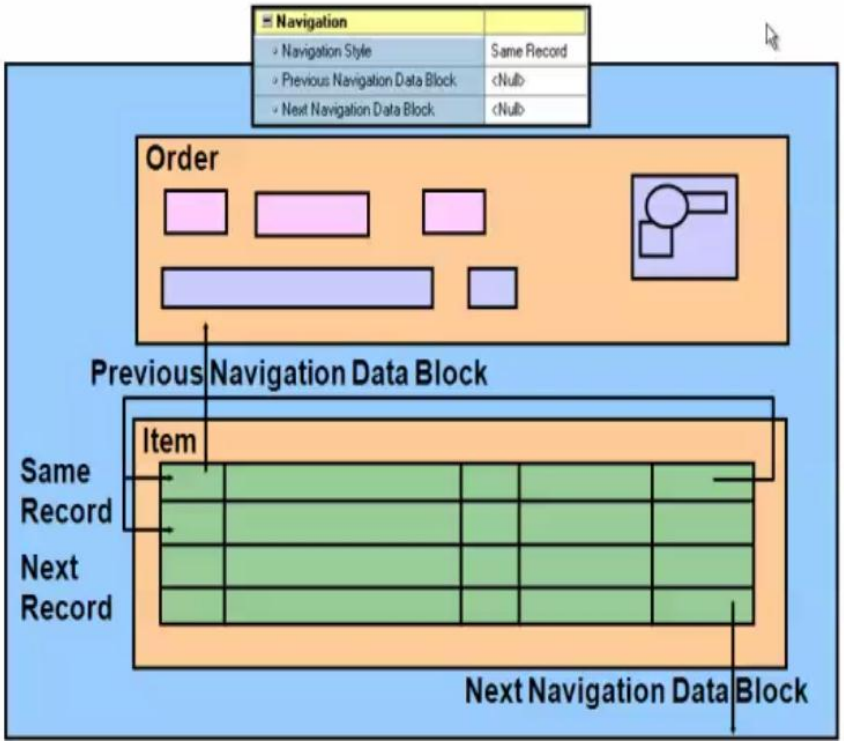
Controlling Data Block Behavior and Appearance

Data Block Property Groups:

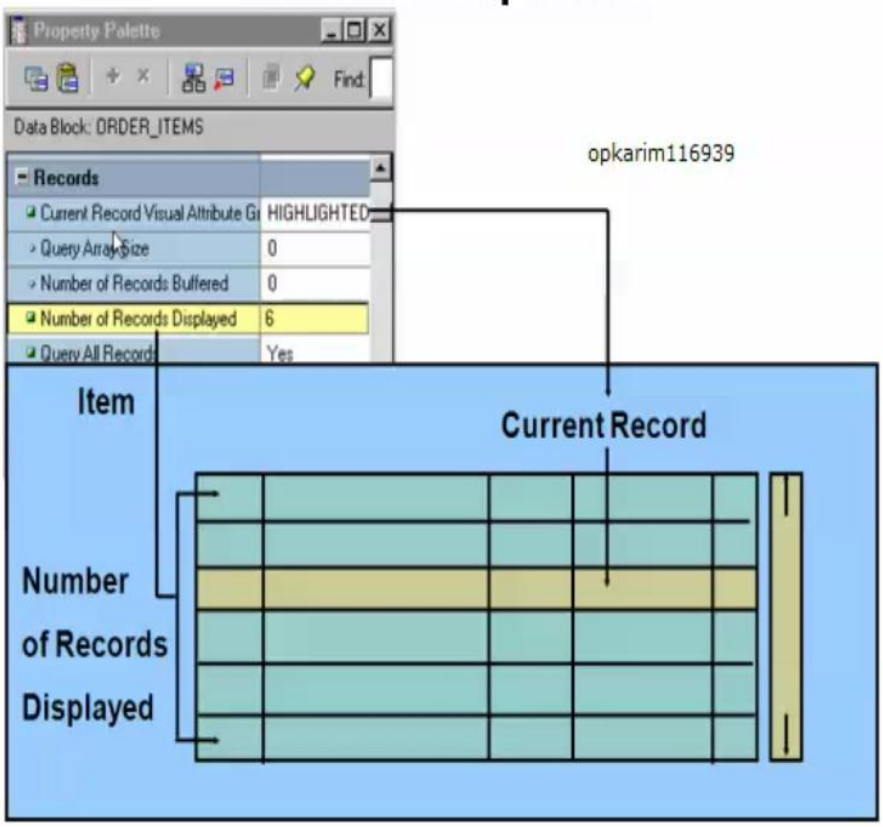
- General
- Navigation
- Records
- Database
- Advanced Database
- Scrollbar
- Visual Attributes
- Color
- International



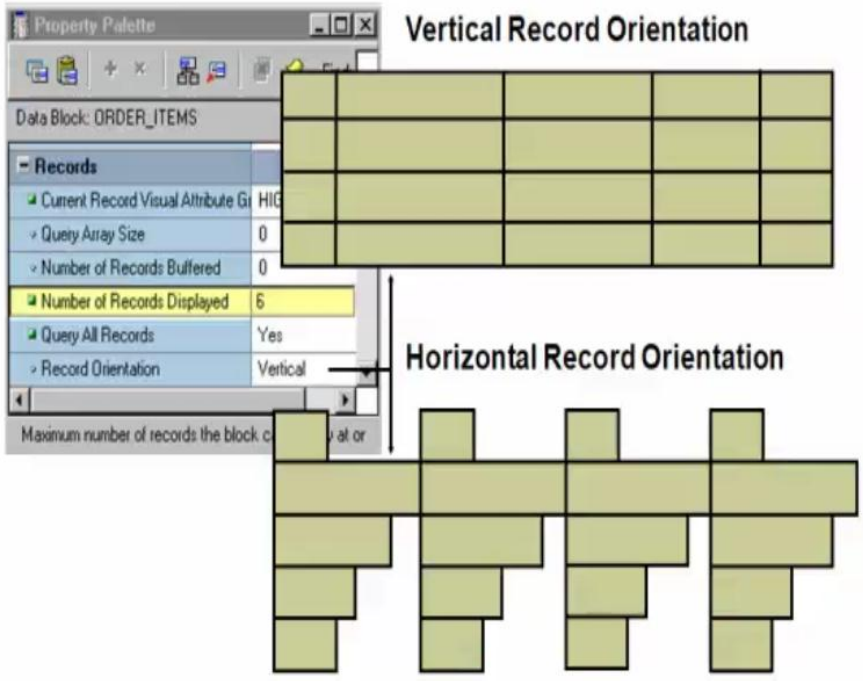
Navigation Properties



Records Properties



Records Properties



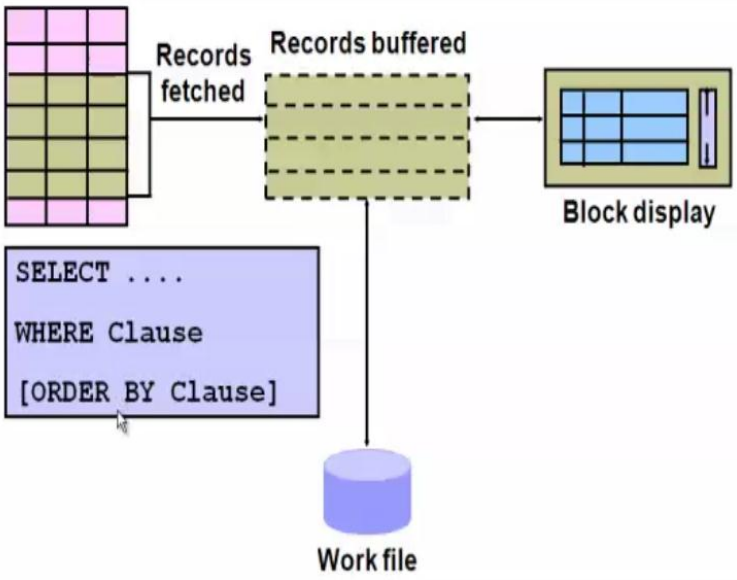
Database Properties

Use properties in the Database group to control:

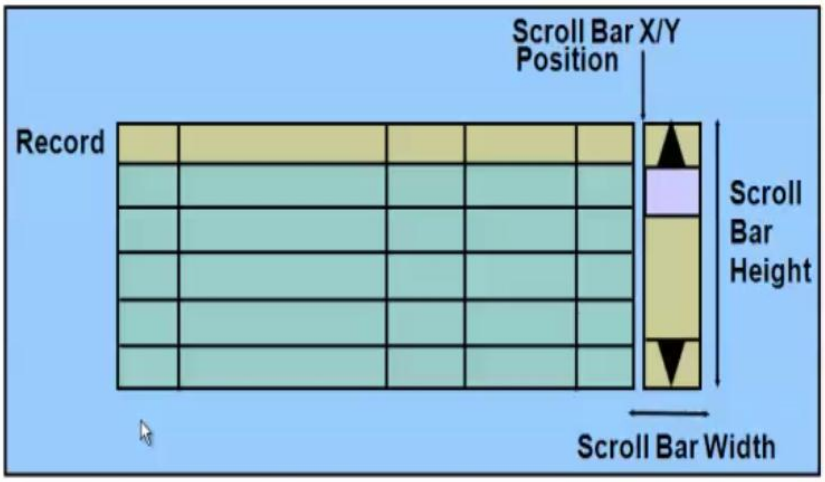
- Type of block—data or control block
- Query, insert, update, and delete operations on the data block
- Data block's data source
- Query search criteria and default sort order
- Maximum query time
- Maximum number of records fetched

Database	
Database Data Block	Yes
Enforce Primary Key	No
Query Allowed	Yes
Query Data Source Type	Table
Query Data Source Name	ORDERS
Query Data Source Columns	
Query Data Source Arguments	
Alias	
Include REF Item	No
WHERE Clause	
ORDER BY Clause	order_id
Optimizer Hint	
Insert Allowed	Yes
Update Allowed	Yes
Locking Mode	Automatic
Delete Allowed	Yes
Key Mode	Automatic
Update Changed Columns Only	No
Enforce Column Security	No
Maximum Query Time	0
Maximum Records Fetched	0

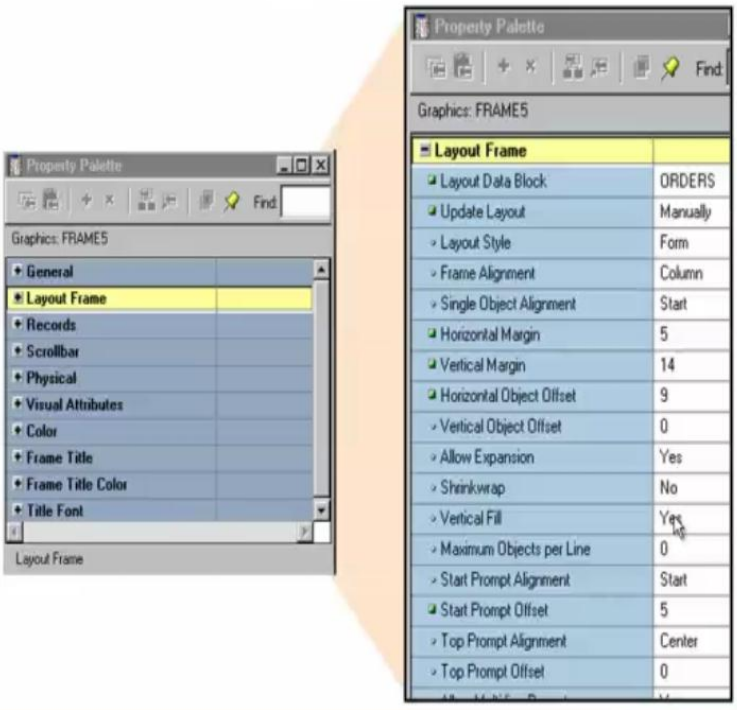
Database Properties



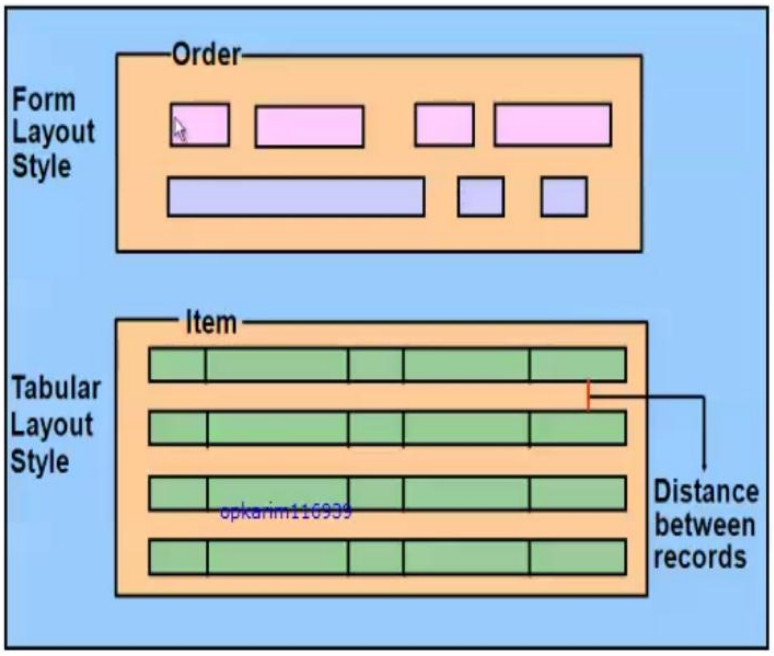
Scroll Bar Properties



Controlling Frame Properties

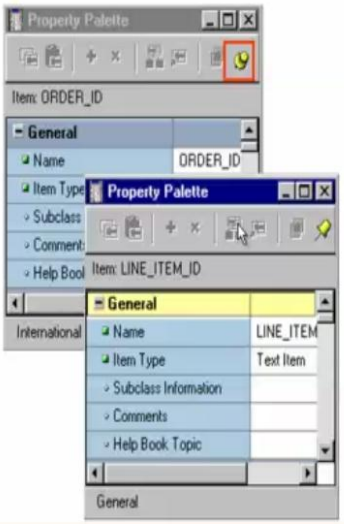


Controlling Frame Properties

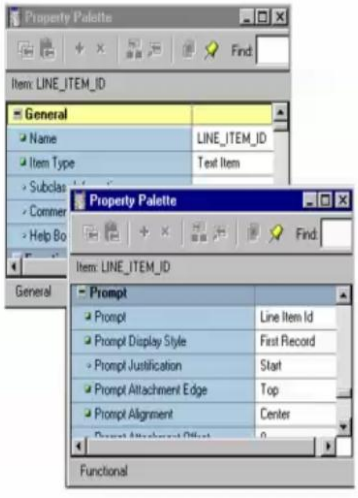


Displaying Multiple Property Palettes

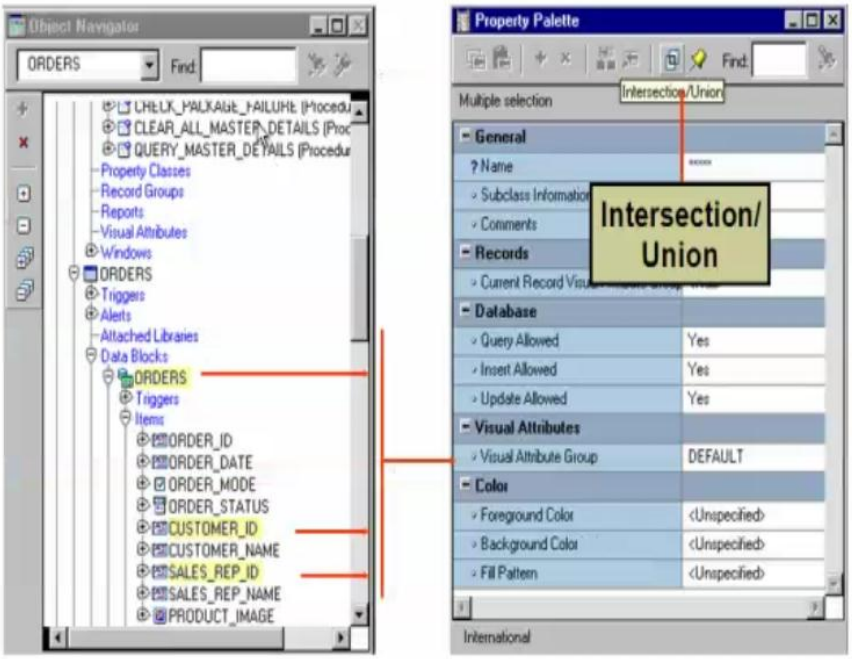
Two Palettes for Two Items:



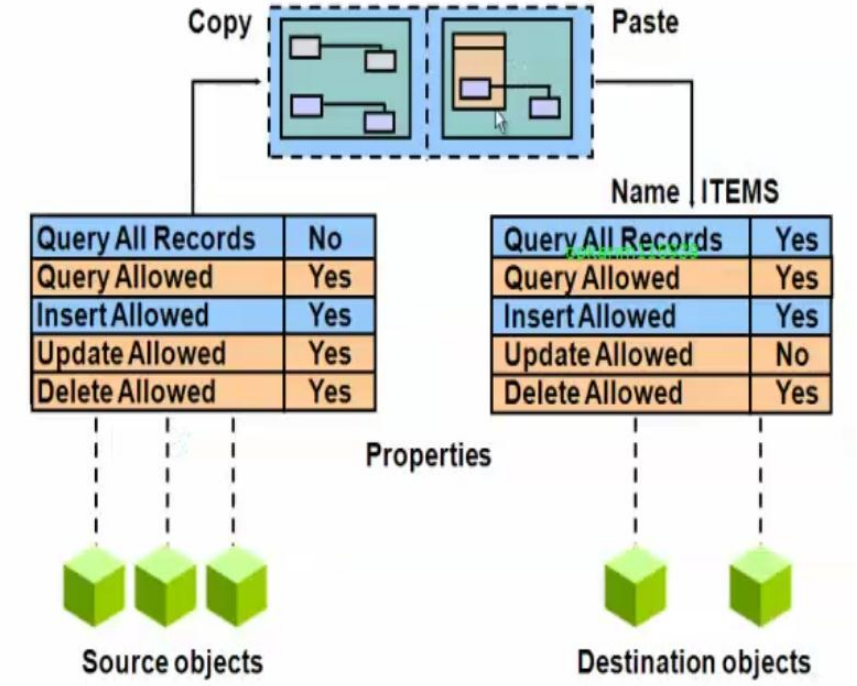
Two Palettes for One Item:



Setting Properties on Multiple Objects



Copying Properties



Creating a Control Block

- Click the Data Blocks node
- Click the Create icon

  OR

  Select Edit > Create.
- Select the "Build a new data block manually" option in the New Data Block dialog box.





Deleting a Data Block

- Select a data block for deletion
- Click the Delete icon

  OR

  Press [Delete]
- Click Yes in the alert box.





Summary

In this lesson, you should have learned that:

- The Property Palette:
  - Contains property names and values that enable you to modify Forms objects
  - Has tools to search for properties, inherit properties, expand or collapse property categories, and pop up lists and dialog boxes for various properties
  - Shows different icons for default, changed, inherited, and overridden properties
- Block properties control the behavior and appearance of data blocks
- Frame properties control how block items are arranged
- You can create blocks that do not directly correspond to database tables by choosing to create the block manually rather than using the Data Block Wizard
- Deleting a data block deletes all of its components

Creating Additional Input Items

Objectives

- After completing this lesson, you should be able to do the following:
  - Identify the item types that allow input
  - Create a check box
  - Create a list item
  - Create a radio group

Input Items Overview

- What are input items?
  - Item types that accept user input include:
    - Check boxes
    - List items
    - Radio groups
  - Input items enable insert, update, delete, and query.

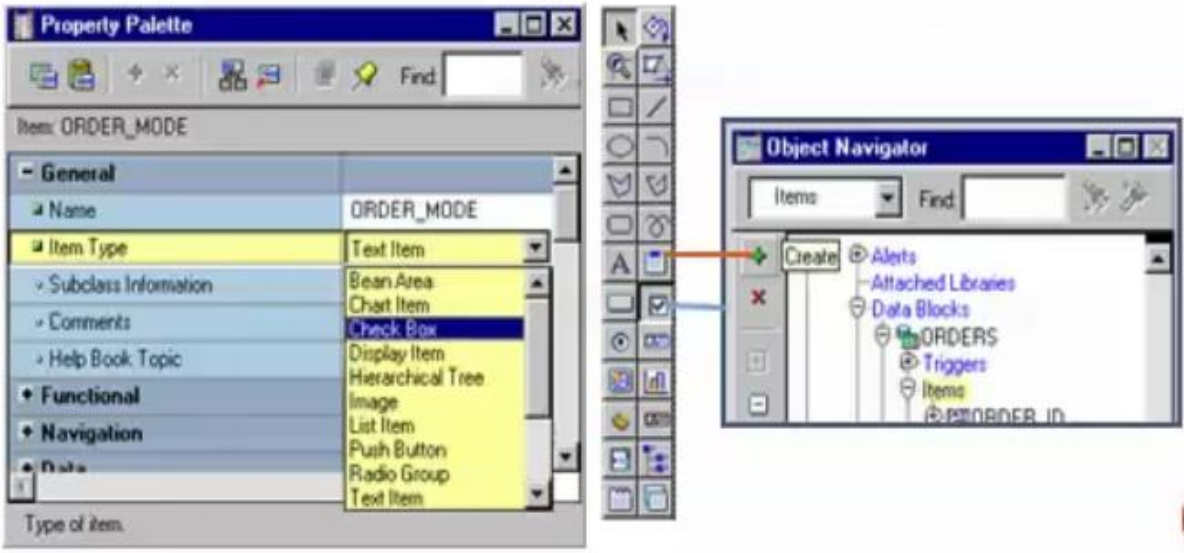
Check Boxes Overview

- What Are Check Boxes?
  - Two-state interface object:
    - Checked
    - Unchecked
  - Not limited to two values



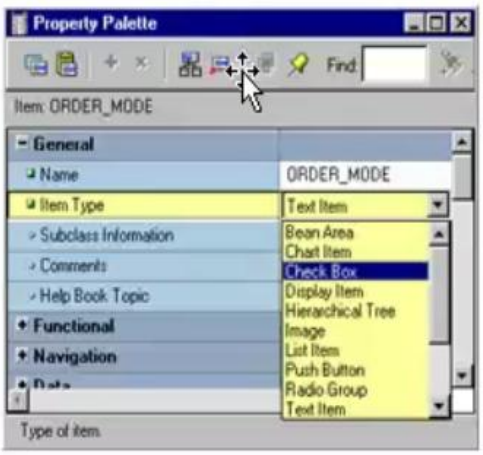
Creating a Check Box

- Convert an existing item.
- Use the Check Box tool in the Layout Editor.
- Use the Create icon in the Object Navigator.



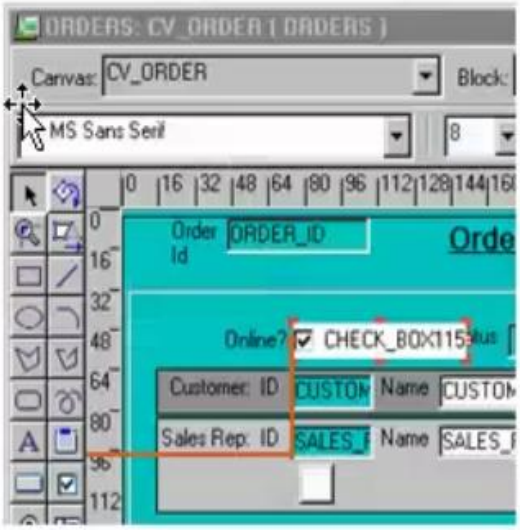
Converting an Existing Item into a Check Box

Convert text item
to check box

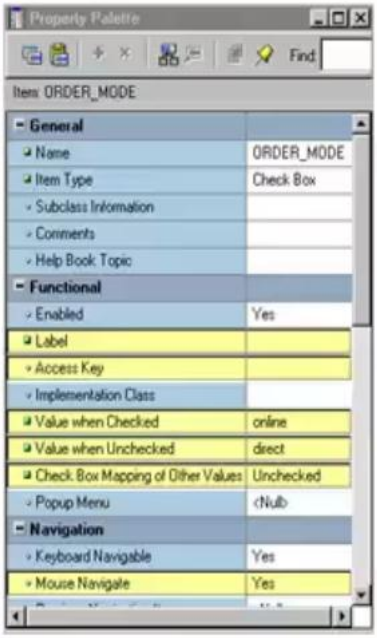


Creating a Check Box in the Layout Editor

Use check box tool
in Layout Editor

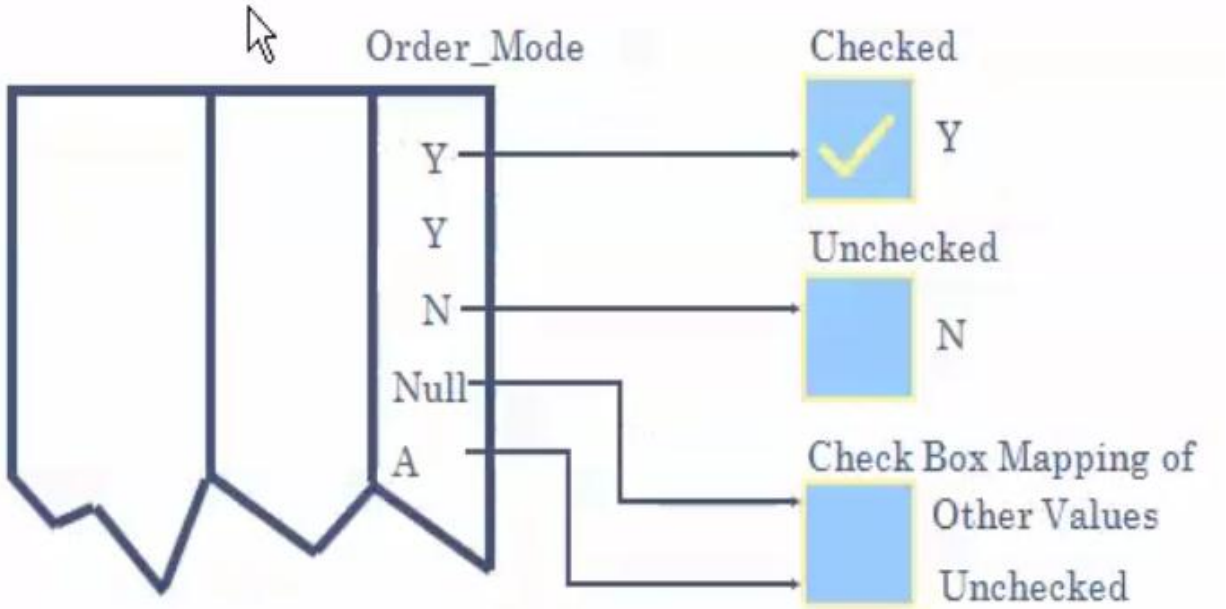


Setting Check Box Properties



- Data Type
- Label
- Access Key
- Value When Checked
- Value When Unchecked
- Check Box Mapping of Other Values
- Mouse Navigate

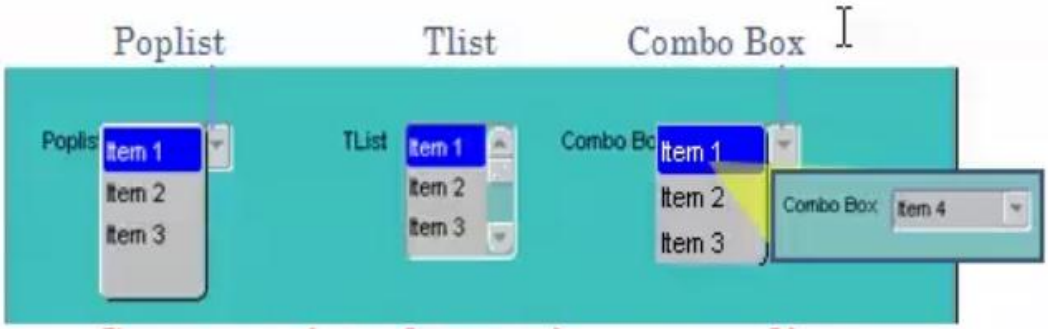
Check Box Mapping of Other Values



List Items Overview

What Are List Items?

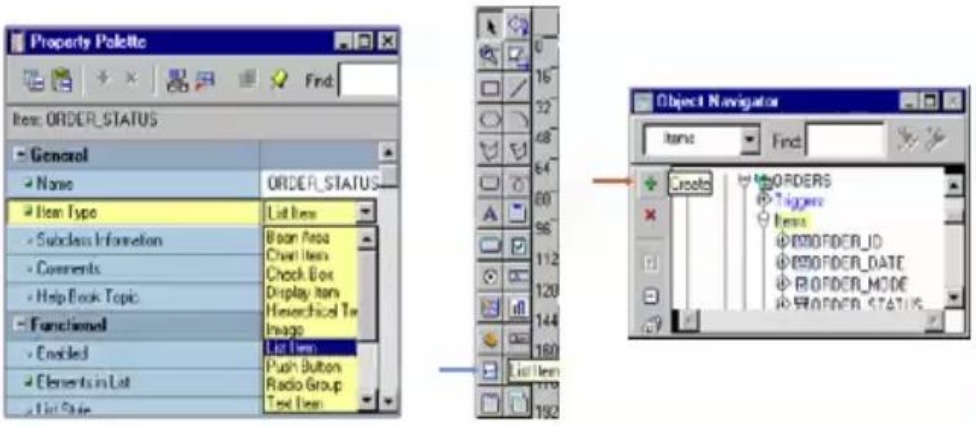
- Set of mutually exclusive choices, each representing a different value
- Three list styles available:



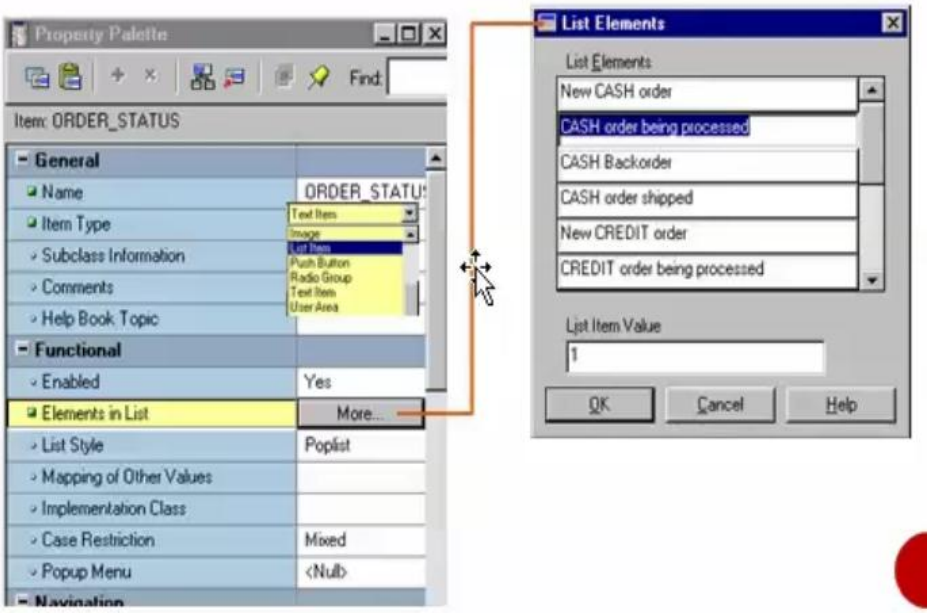
- Space-saving alternative to a radio group
- Smaller-scale alternative to an LOV

Creating a List Item

- Convert an existing item.
- Use the List Item tool in the Layout Editor.
- Use the Create icon in the Object Navigator.



Converting an Existing Item into a List Item



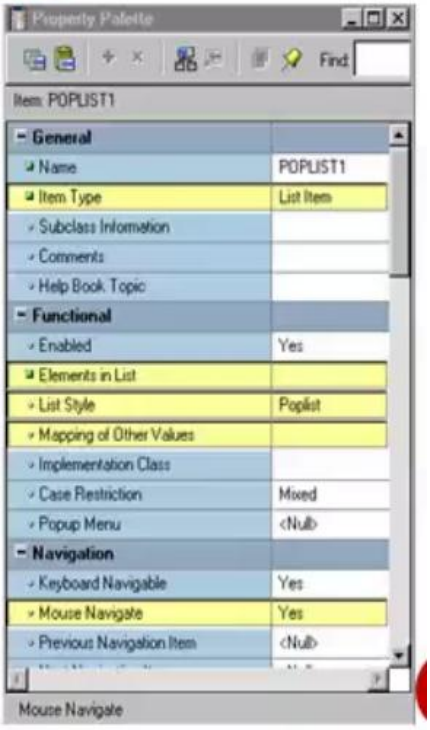
Creating a List Item in the Layout Editor

Use list item tool
in Layout Editor

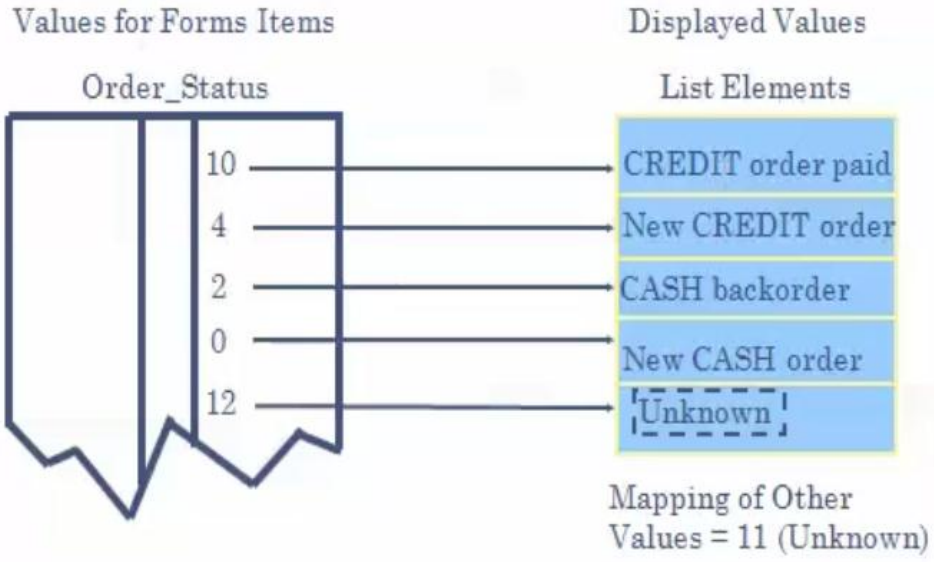


Setting List Item Properties

- Elements in List:
  - o List elements
  - o List item value
- List Style
- Mapping of Other Values
- Mouse Navigate

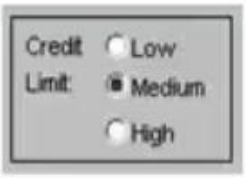


List Item Mapping of Other Values



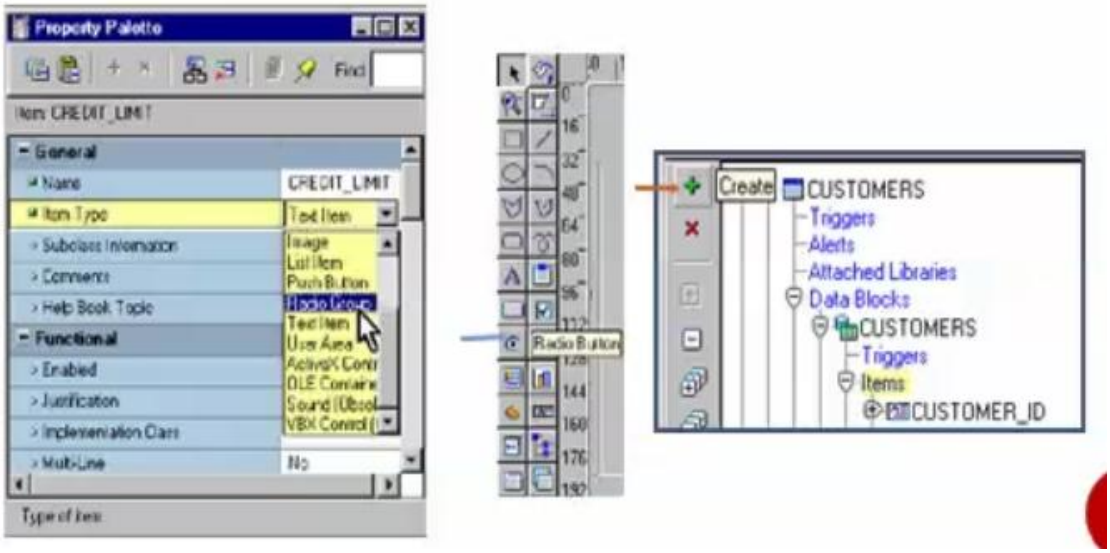
Radio Groups Overview

- What are radio groups?
  - Set of mutually exclusive radio buttons, each representing a value
  - Use:
    - To display two or more static choices
    - As an alternative to a list item
    - As an alternative to a check box

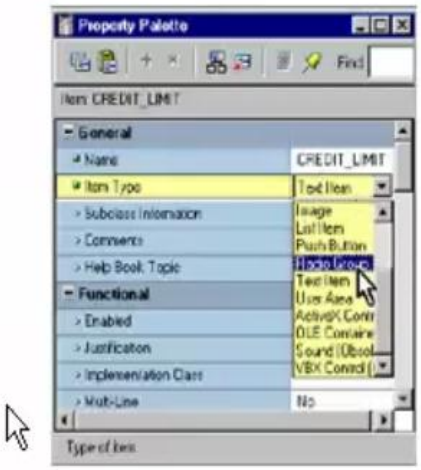


Creating a Radio Group

- Convert an existing item.
- Create a new radio button in the Layout Editor.
- Use the Create icon in the Object Navigator.



Converting Existing Item to Radio Group

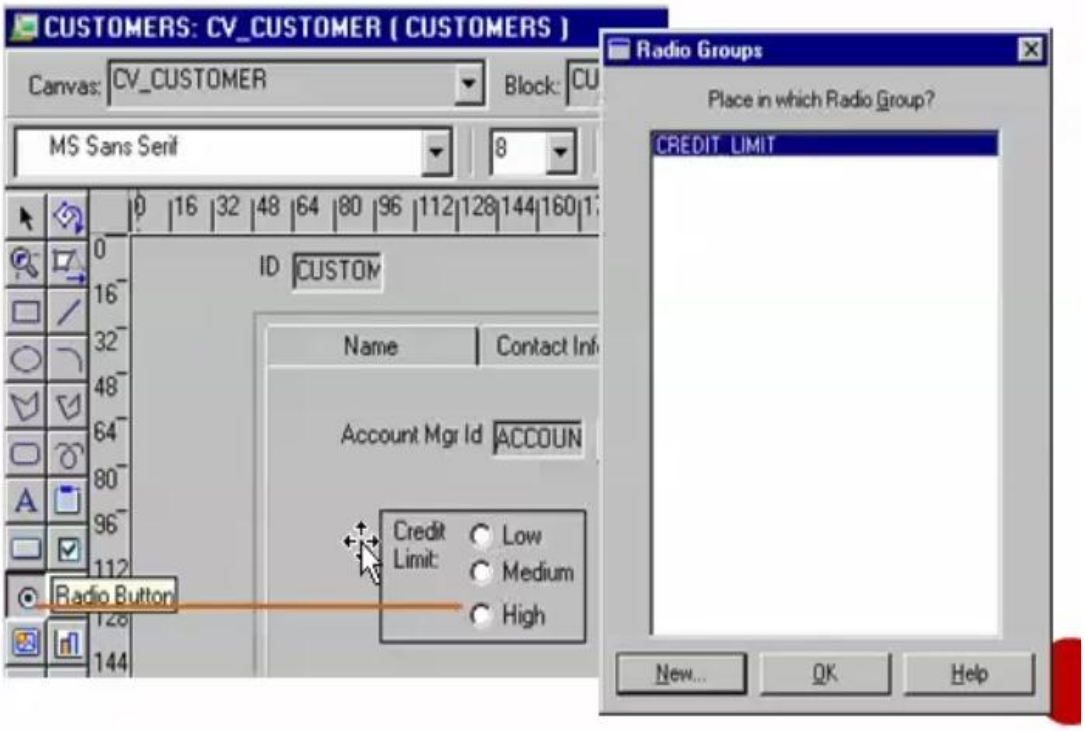


Change Item Type and set other properties



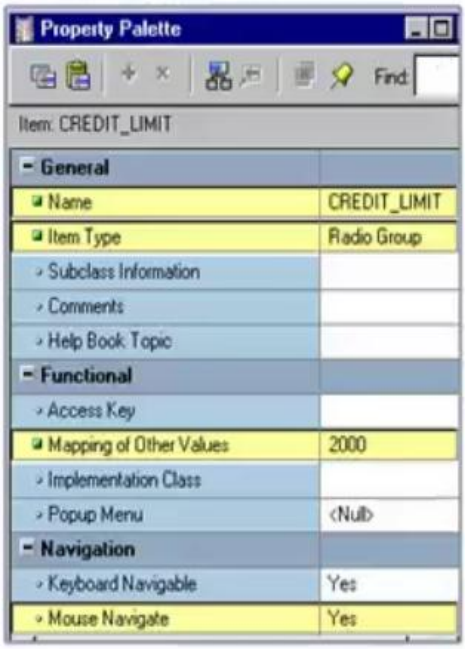
Create radio buttons for the radio group

Creating Radio Group in Layout Editor

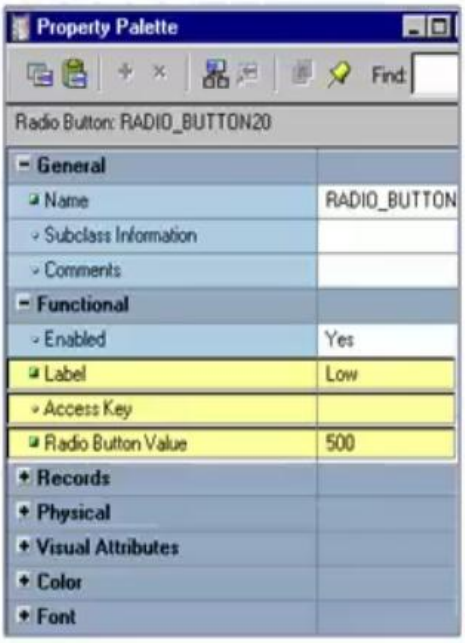


Setting Radio Properties

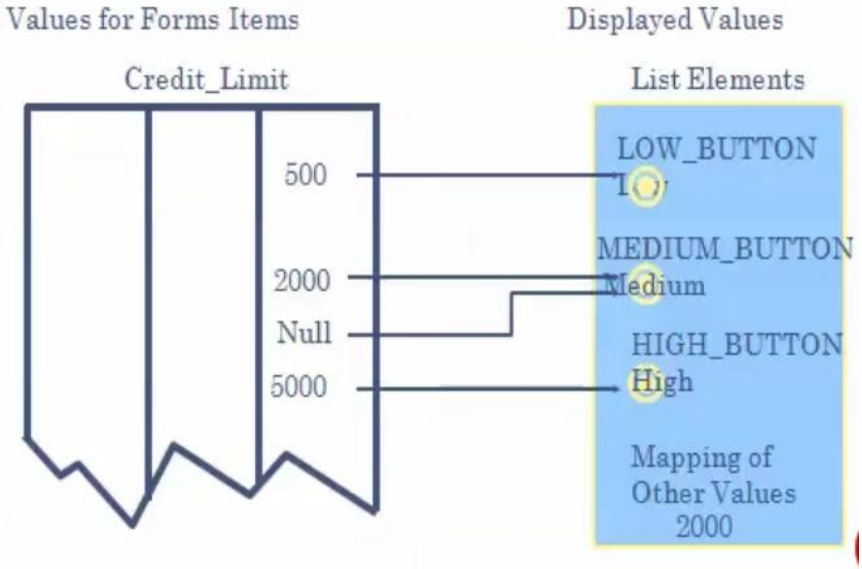
- Radio group:



- Radio button:



Radio Group Mapping of Other Values



Creating Noninput Items

Objectives

- After completing this lesson, you should be able to do the following:
  - Identify item types that do not allow input
  - Create a display item
  - Create a button
  - Create a calculated item

Noninput Items Overview

- Item types that do not accept direct user input include:
  - Display items
  - Buttons
  - Calculated items

Display Items

- Display items:
  - Are similar to text items.
  - Cannot:
    - Be edited
    - Be queried
    - Be navigated to
    - Accept user input
  - Can display:
    - Nonbase table information
    - Derived values



Creating a Display Item



Push Buttons

- Push buttons:

opkarim116939

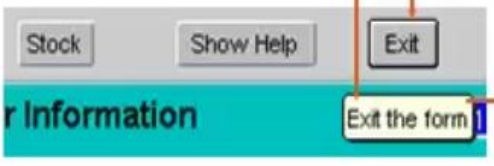


  - Cannot display or represent data
  - Are used to initiate an action
  - Display as:
    - Text button
    - Iconic



Setting Push Button Properties

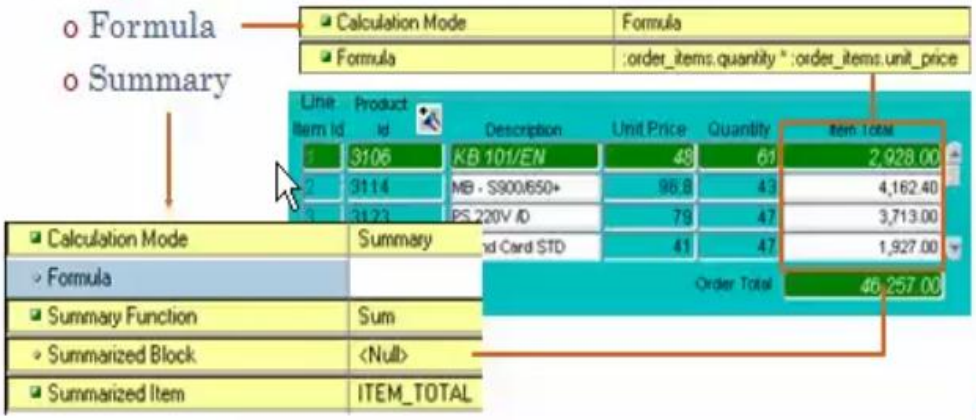
- Label
- Iconic
- Icon Filename
- Default Button
- Mouse Navigate
- Tooltip
- Tooltip Visual Attribute Group





Calculated Items

- What are calculated items?
  - They accept item values that are based on calculations.
  - They are read-only.
  - They can be expressed as:
    - Formula
    - Summary



Creating a Calculated Item by Setting Properties

Calculation Mode	Formula
Formula	:order_items.quantity * :order_items.unit_price

- Formula
  - A calculated item value is the result of a horizontal calculation.
  - It involves bind variables.
- Summary
  - A calculated item value is a vertical calculation.
  - A summary is performed on values of a single item over all rows in a block.

Calculation Mode	Summary
Formula	
Summary Function	Sum
Summarized Block	<Null>
Summarized Item	ITEM_TOTAL

Rules for Summary Items

When creating calculated items based on a summary, observe the following rules:

The summary item must reside in the same block as the summarized item, or in a control block whose Single Record property is set to Yes.

The summarized item must reside in a control block, or in a data block whose Query All Records property or the Precompute Summaries property is set to Yes.

Note: This ensures that records fetched in the block and the summarized value are consistent. Otherwise, another user may possibly update a record that has not been fetched yet.

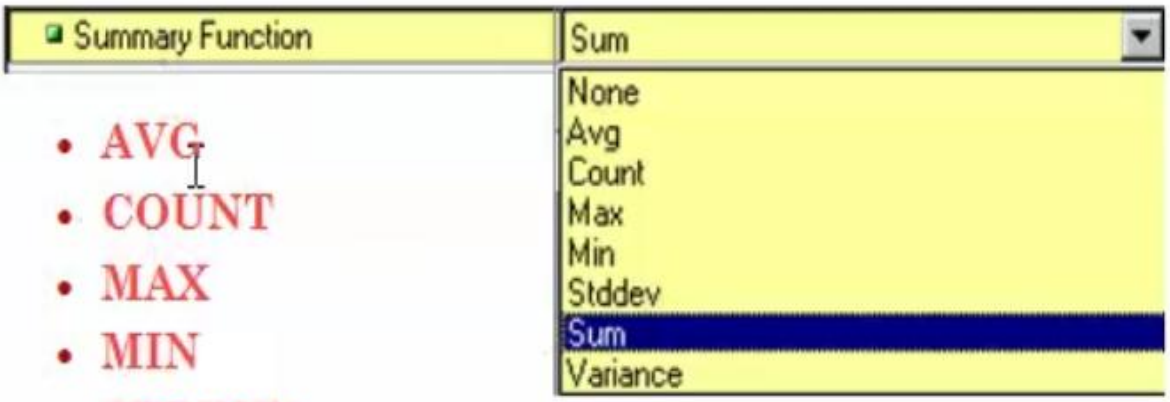
Set the Data Type property for a summary item to Number, unless the summary function is Max or Min, in which case the datatype must mirror that of its associated summarized item. For example, a calculated item that displays the most recent (maximum) date in the `HIRE_DATE` column must have a datatype of Date.

If the summarized item values are based on a formula, the summarized item must reside in a block whose Query All Records property is set to Yes.

Setting Item Properties for the Calculated Item

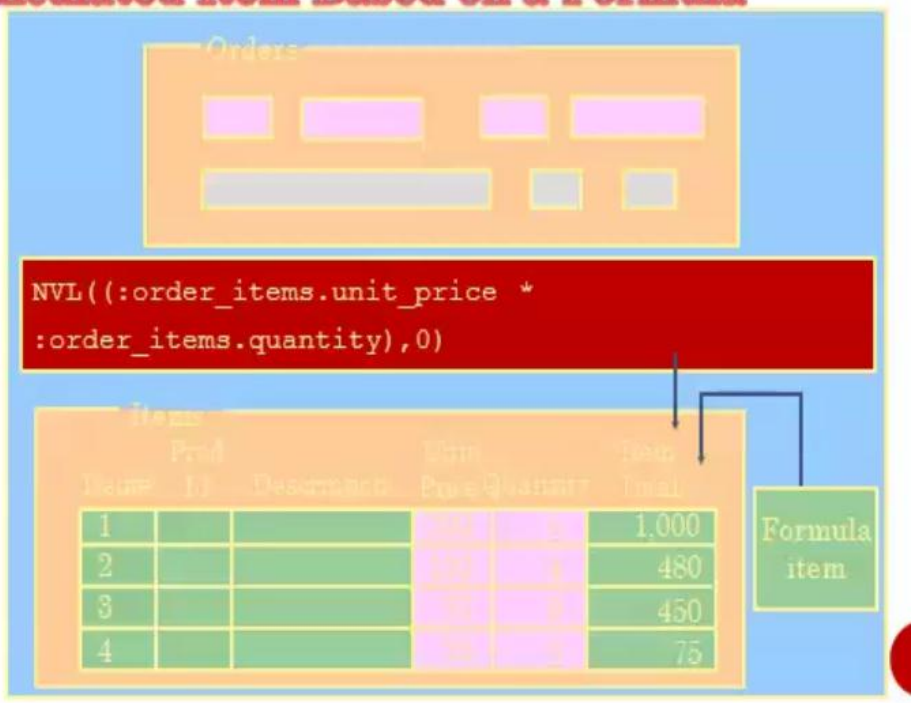
- Formula
  - Calculation Mode
  - Formula
- Summary
  - Calculation Mode
  - Summary Function
  - Summarized Block
  - Summarized Item

Summary Functions



- AVG
- COUNT
- MAX
- MIN
- STDDEV
- SUM
- VARIANCE

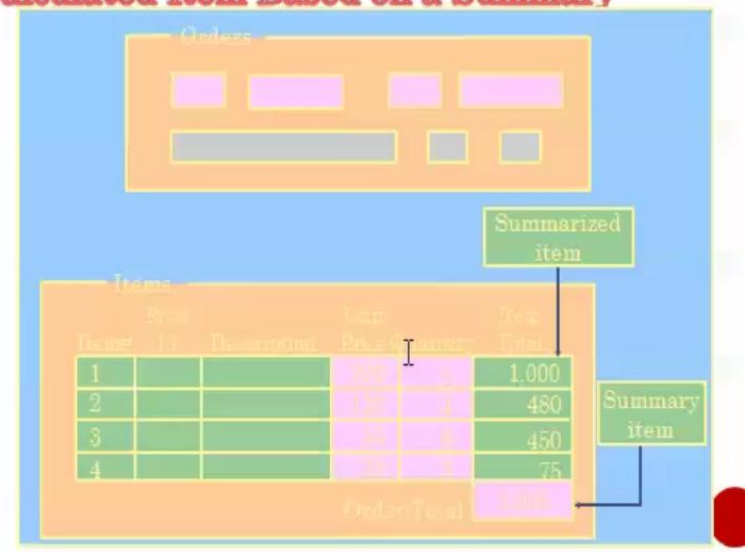
Calculated Item Based on a Formula



Rules for Calculated Item Formulas

- Create calculated item formulas according to the following rules:
  - A formula item must not invoke restricted built-ins.
  - A formula item cannot execute any DML statements.
  - Do not terminate a PL/SQL expression with a semicolon.
  - Do not enter a complete PL/SQL statement in assignment expressions.

Calculated Item Based on a Summary

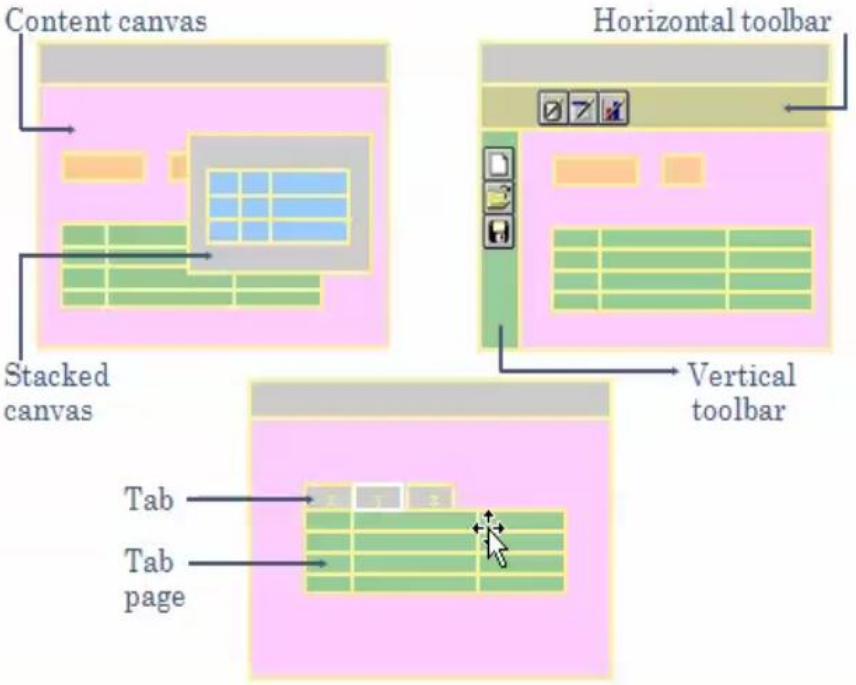


Rules for Summary Items

- Summary item must reside in:
  - The same block as the summarized item
  - A control block with Single Record property set to Yes
- Summarized item must reside in:
  - A data block with Query All Records property or Precompute Summaries property set to Yes
  - A control block
- Datatype of summary item must be Number, unless using MAX or MIN

Working with Other Canvas Types

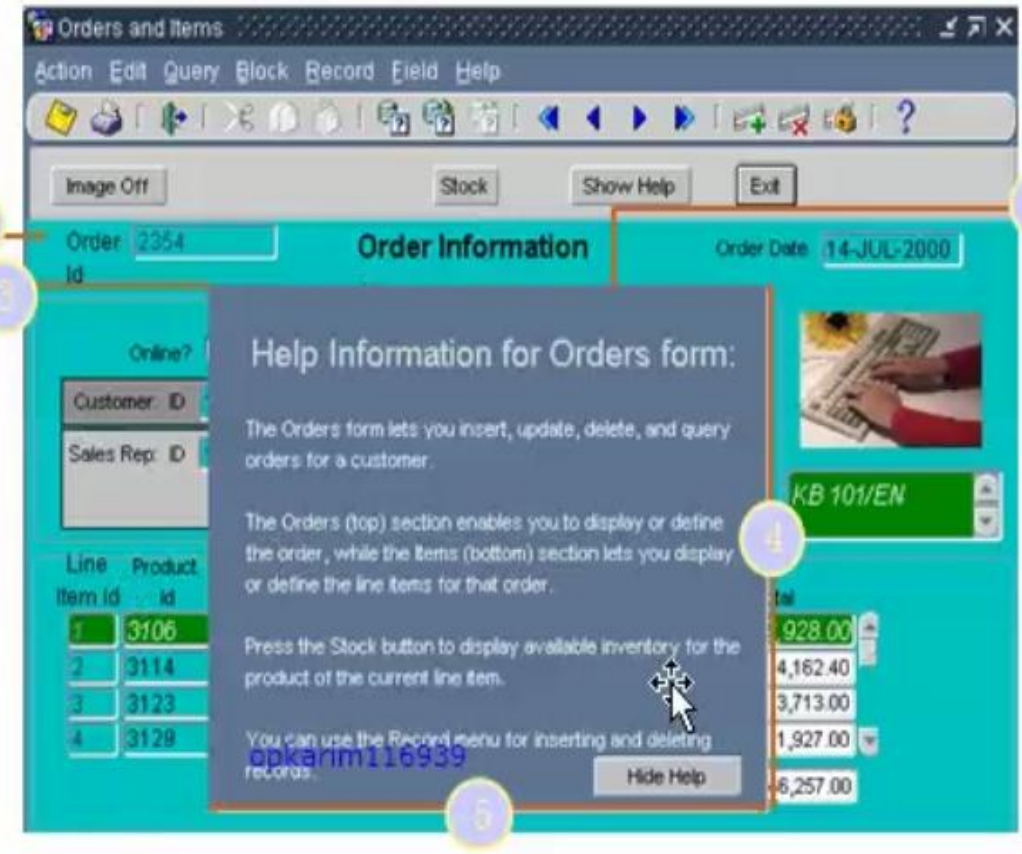
Overview of Canvas Types



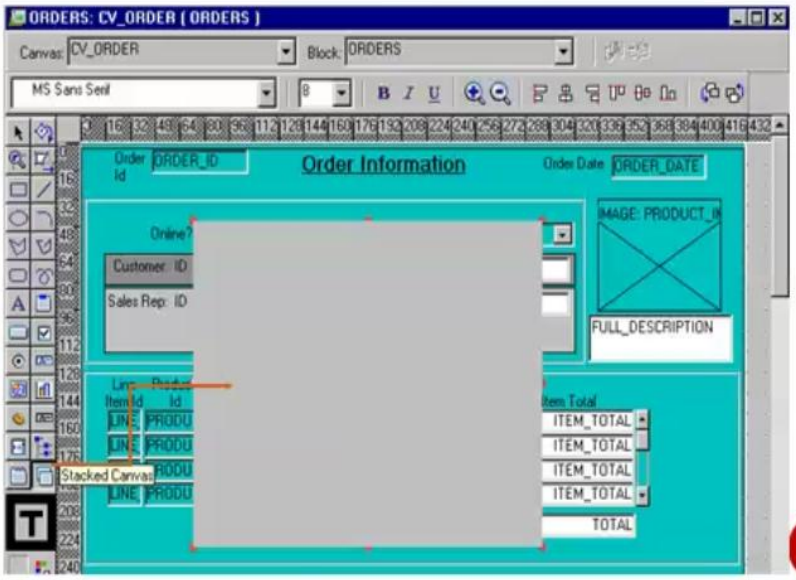
The Stacked Canvas

- Displayed on top of a content canvas
- Shares a window with a content canvas
- Size:
  - Usually smaller than the content canvas in the same window
  - Determined by viewport size
- Created in:
  - Layout Editor
  - Object Navigator

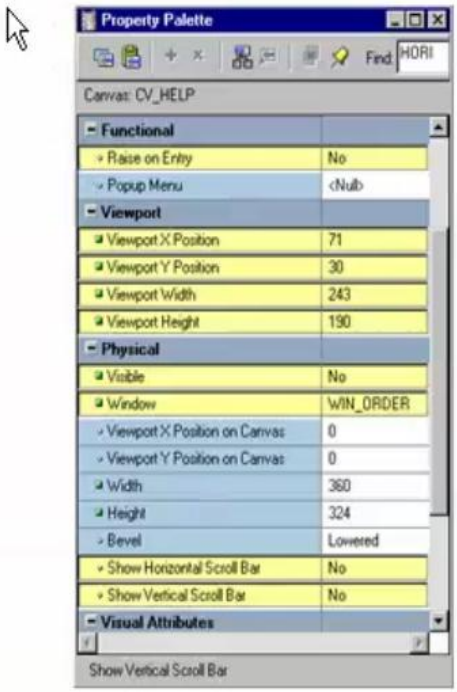
The Stacked Canvas



Creating a Stacked Canvas

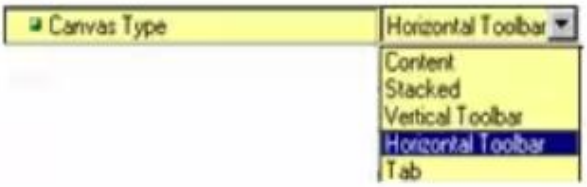


Setting Stacked Canvas Properties

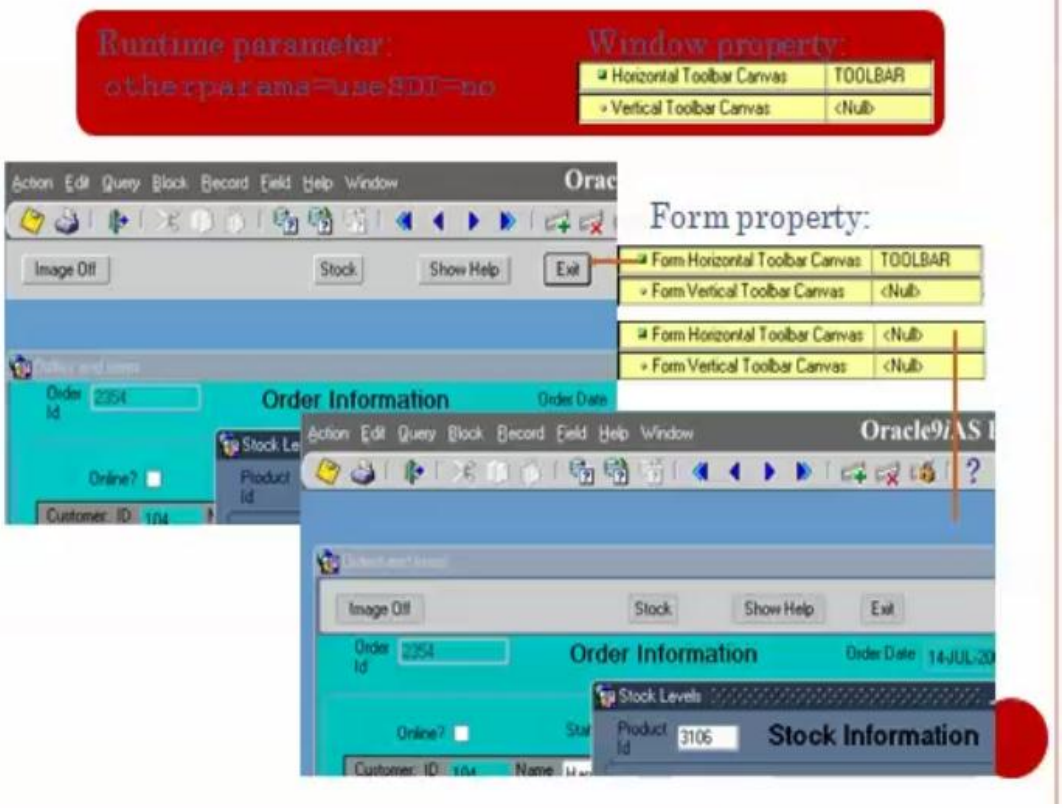


The Toolbar Canvas

- Special type of canvas for tool items
- Two types:
  - o Vertical toolbar
  - o Horizontal toolbar
- Provide:
  - o Standard look and feel
  - o Alternative to menu or function key operation



The MDI Toolbar

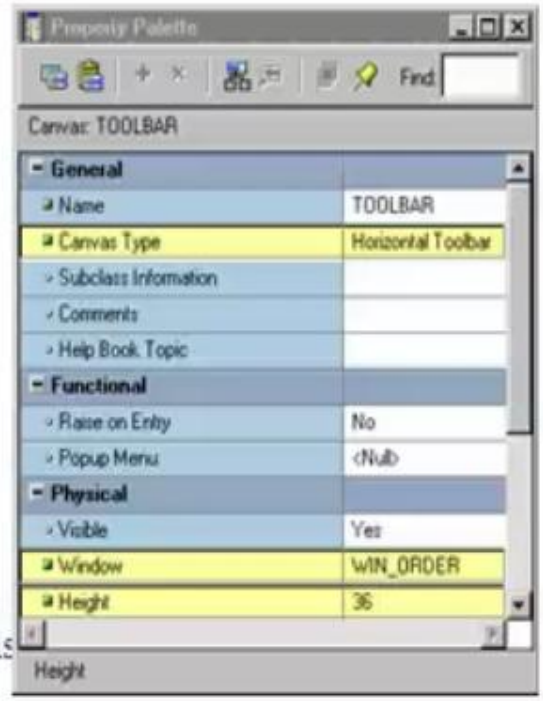


Creating a Toolbar Canvas

1. Create:
   - o Click Create in Object Navigator
   - o Change Canvas Type
   - o Set other properties as required
2. Add functionality
3. Resize the canvas (not the view)
4. Assign to window and/or form

Setting Toolbar Properties

- Canvas properties:
  - o Canvas Type
  - o Window
  - o Width or Height
- Window properties:
  - o Horizontal Toolbar Canvas
  - o Vertical Toolbar Canvas
- Form Module properties:
  - o Form Horizontal Toolbar Canvas
  - o Form Vertical Toolbar Canvas



The Tab Canvas



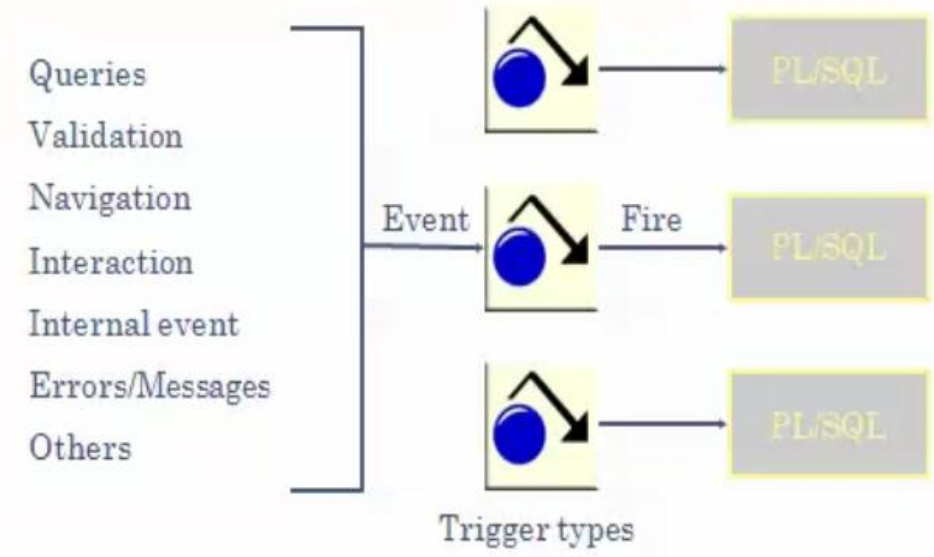
- Enables you to organize and display related information on separate tabs
- Consists of one or more tab pages
- Provides easy access to data

Introduction to Triggers

Objectives

- After completing this lesson, you should be able to do the following:
  - Define triggers
  - Identify the different trigger categories
  - Plan the type and scope of triggers in a form
  - Describe the properties that affect the behavior of a trigger

Trigger Overview



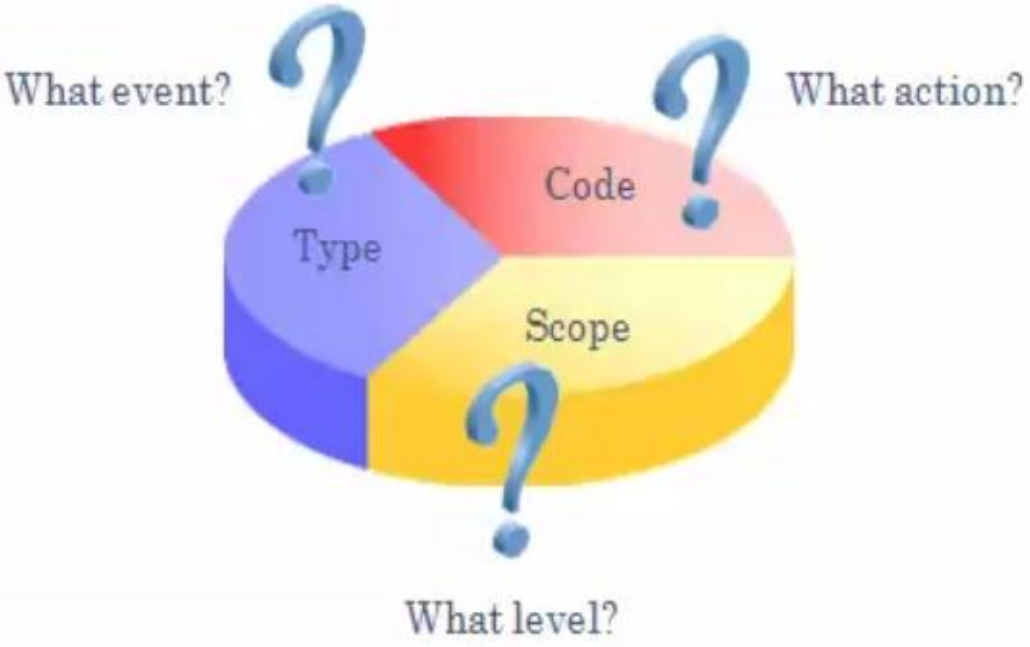
Trigger types

Which trigger would you use to perform complex calculations after a user enters data into an item?

Grouping Triggers into Categories

- Triggers may be grouped into functional categories:
  - Block processing triggers
  - Interface event triggers
  - Master-detail triggers
  - Message handling triggers
  - Navigational triggers
  - Query-time triggers
  - Transactional triggers
  - Validation triggers
- Triggers may be grouped into categories based on name:
  - When-Event triggers
  - On-Event triggers
  - Pre-Event triggers
  - Post-Event triggers
  - Key triggers

Defining Trigger Components

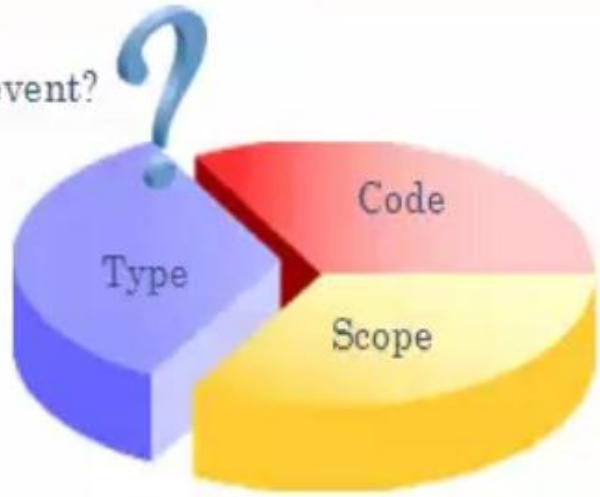


Trigger Type

- Pre-
- Post-
- When-
- On-
- Key-
- User-named



What event?

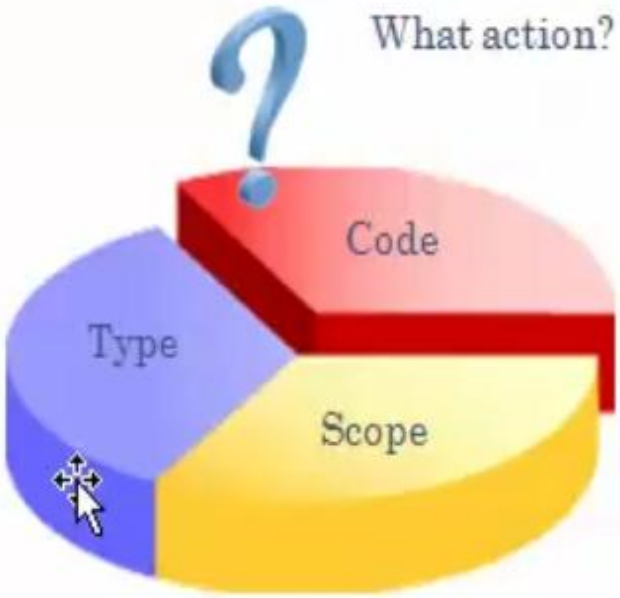


Trigger Type

(User-named)	KEY-NXTKEY	ON-UPDATE	WHEN-CREATE-RECORD
KEY-CLRBLK	KEY-NXTREC	POST-BLOCK	WHEN-CUSTOM-ITEM-EVENT
KEY-CLRFRM	KEY-NXTSET	POST-CHANGE	WHEN-DATABASE-RECORD
KEY-CLRREC	KEY-OTHERS	POST-DATABASE-COMMIT	WHEN-FORM-NAVIGATE
KEY-COMMIT	KEY-PREV-ITEM	POST-DELETE	WHEN-IMAGE-ACTIVATED
KEY-CQUERY	KEY-PRINT	POST-FORM	WHEN-IMAGE-PRESSED
KEY-CREREC	KEY-PRVBLK	POST-FORMS-COMMIT	WHEN-LIST-ACTIVATED
KEY-DELREC	KEY-PRVREC	POST-INSERT	WHEN-LIST-CHANGED
KEY-DOWN	KEY-SCRDOWN	POST-LOGON	WHEN-MOUSE-CLICK
KEY-DUP-ITEM	KEY-SCRUP	POST-LOGOUT	WHEN-MOUSE-DOUBLECLICK
KEY-DUPREC	KEY-UP	POST-QUERY	WHEN-MOUSE-DOWN
KEY-EDIT	KEY-UPDREC	POST-RECORD	WHEN-MOUSE-ENTER
KEY-ENTER	ON-CHECK-DELETE-MASTER	POST-SELECT	WHEN-MOUSE-LEAVE
KEY-ENTQRY	ON-CHECK-UNIQUE	POST-TEXT-ITEM	WHEN-MOUSE-MOVE
KEY-EXEQRY	ON-CLOSE	POST-UPDATE	WHEN-MOUSE-UP
KEY-EXIT	ON-COLUMN-SECURITY	PRE-BLOCK	WHEN-NEW-BLOCK-INSTANCE
KEY-F0	ON-COMMIT	PRE-COMMIT	WHEN-NEW-ITEM-INSTANCE
KEY-F1	ON-COUNT	PRE-DELETE	WHEN-NEW-RECORD-INSTANCE
KEY-F2	ON-DELETE	PRE-INSERT	WHEN-RADIO-CHANGED
KEY-F3	ON-FETCH	PRE-LOGON	WHEN-REMOVE-RECORD
KEY-F4	ON-INSERT	PRE-LOGOUT	WHEN-TAB-PAGE-CHANGED
KEY-F5	ON-LOCK	PRE-POPUP-MENU	WHEN-TIMER-EXPIRED
KEY-F6	ON-LOGON	PRE-QUERY	WHEN-TREE-NODE-ACTIVATED
KEY-F7	ON-LOGOUT	PRE-RECORD	WHEN-TREE-NODE-EXPANDED
KEY-F8	ON-MESSAGE	PRE-SELECT	WHEN-TREE-NODE-SELECTED
KEY-F9	ON-POPULATE-DETAILS	PRE-TEXT-ITEM	WHEN-VALIDATE-ITEM
KEY-HELP	ON-ROLLBACK	PRE-UPDATE	WHEN-VALIDATE-RECORD
KEY-LISTVAL	ON-SAVEPOINT	WHEN-BUTTON-PRESSED	WHEN-WINDOW-ACTIVATED
KEY-MENU	ON-SELECT	WHEN-CHECKBOX-CHANGED	WHEN-WINDOW-CLOSED
KEY-NEXT-ITEM	ON-SEQUENCE-NUMBER	WHEN-CLEAR-BLOCK	WHEN-WINDOW-DEACTIVATED
KEY-NXTBLK			WHEN-WINDOW-RESIZED

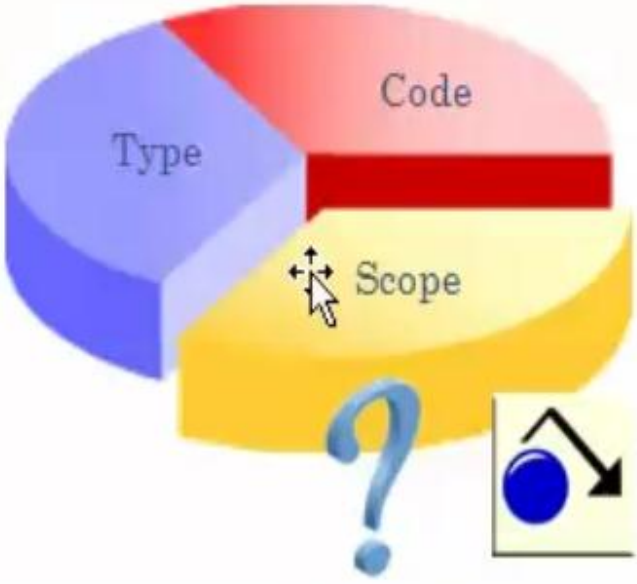
Forms Builder Trigger Types

Trigger Code



- Statements
- PL/SQL
- User subprograms
- Built-in subprograms

Trigger Scope



What level?

- Levels
  - Form
  - Block
  - Item

Navigation

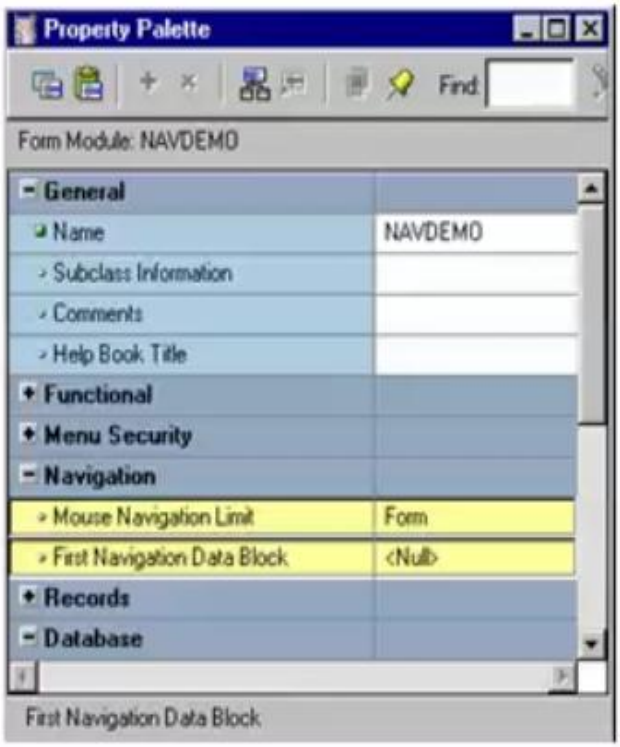
Navigation Overview

- What is the navigational unit?
  - Outside the form
  - Form
  - Block
  - Record
  - Item
- Entering and leaving objects
- What happens if navigation fails?

Using Object Properties to Control Navigation

- Block
  - Navigation Style
  - Previous Navigation Data Block
  - Next Navigation Data Block
- Item
  - Enabled
  - Keyboard Navigable
  - Mouse Navigate
  - Previous Navigation Item
  - Next Navigation Item

Using Object Properties to Control Navigation



- Form module
  - Mouse Navigation Limit
  - First Navigation Data Block

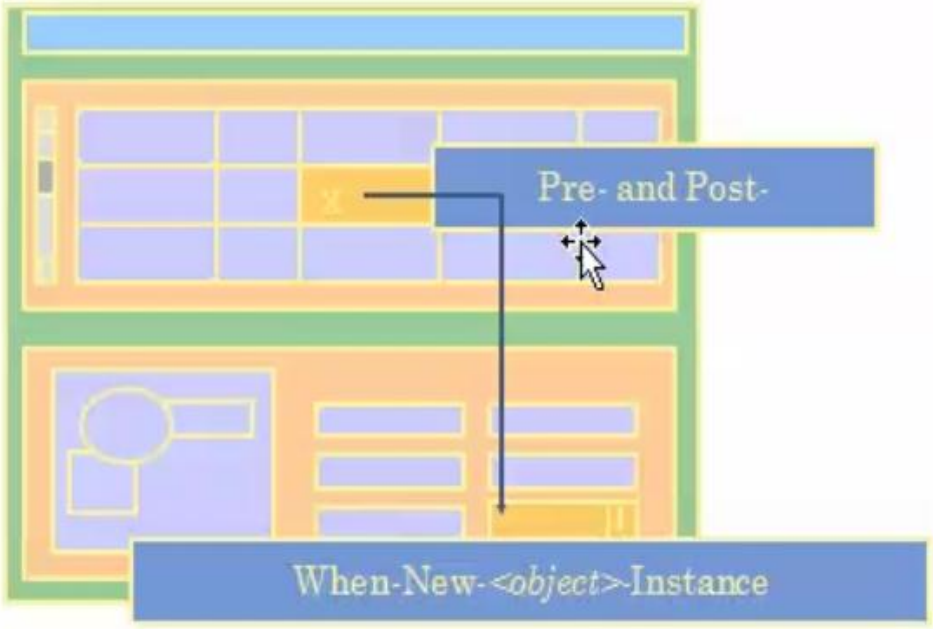
The Pre- and Post-Triggers

- Pre/Post-Form
- Pre/Post-Block
- Pre/Post-Record
- Pre/Post-Text-Item

When-New-<object>-Instance Triggers

- When-New-Form-Instance
- When-New-Block-Instance
- When-New-Record-Instance
- When-New-Item-Instance

Writing Navigation Triggers



Post-Block Trigger Example

- Disabling Stock button when leaving the ORDER_ITEMS block:

```
SET_ITEM_PROPERTY('CONTROL.stock_button',
enabled, property_false);
```

SET_<object>_PROPERTY Examples

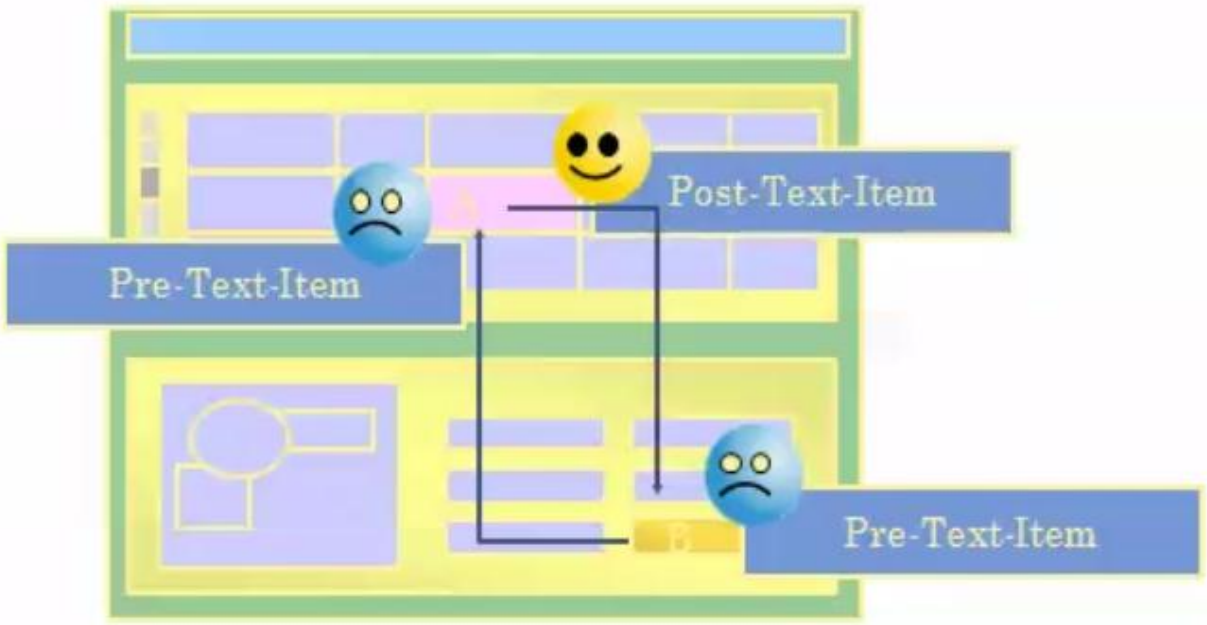
```
SET_FORM_PROPERTY(FIRST_NAVIGATION_BLOCK,
'ORDER_ITEMS');
```

```
SET_BLOCK_PROPERTY('ORDERS', ORDER_BY,
'CUSTOMER_ID');
```

```
SET_RECORD_PROPERTY(3, 'ORDER_ITEMS', STATUS,
QUERY_STATUS);
```

```
SET_ITEM_PROPERTY('CONTROL.stock_button',
ICON_NAME, 'stock');
```

The Navigation Trap



Using Navigation Built-Ins in Triggers

```
GO_FORM
GO_BLOCK
GO_ITEM
GO_RECORD
NEXT_BLOCK
NEXT_ITEM
NEXT_KEY
NEXT_RECORD
```

Using Navigation Built-Ins in Triggers

- When-New-Item-Instance

```
IF CHECKBOX_CHECKED('ORDERS.order_mode') --Online
  THEN                                    -- order
     ORDERS.order_status := 4; --Credit order
     GO_ITEM('ORDERS.order_status');
END IF;
```



- Pre-Text-Item

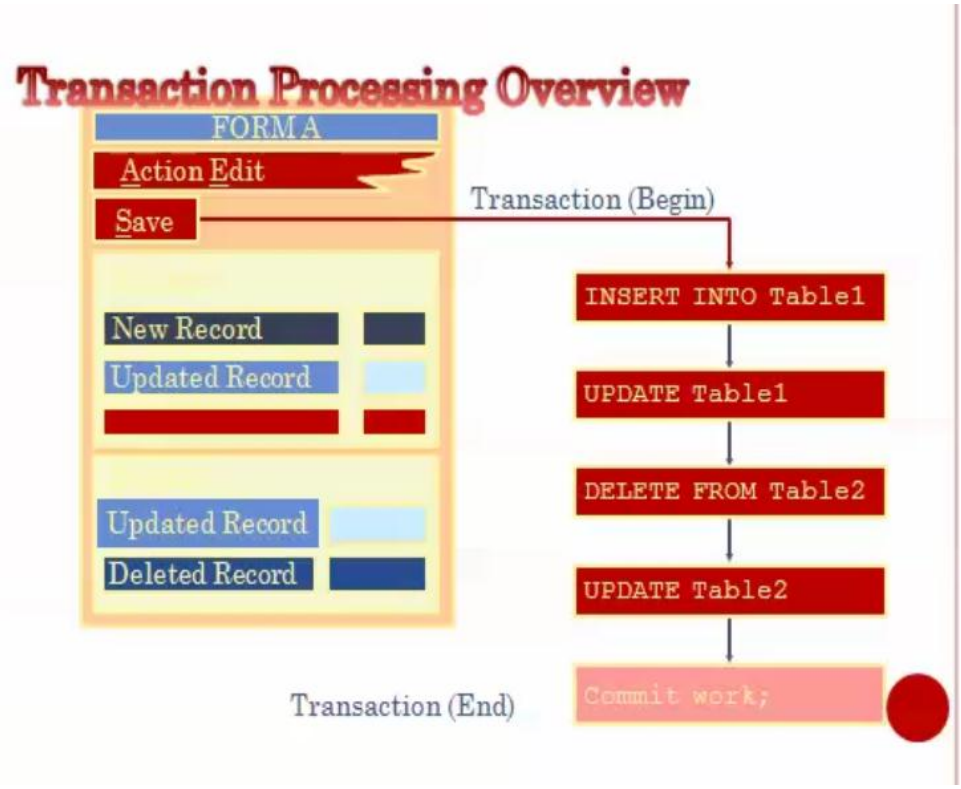
```
IF CHECKBOX_CHECKED('ORDERS.order_mode') --Online
  THEN                                    -- order
     ORDERS.order_status := 4; --Credit order
     GO_ITEM('ORDERS.order_status');
END IF;
```



Transaction Processing

Objectives

- After completing this lesson, you should be able to do the following:
  - Explain the process used by Forms to apply changes to the database
  - Describe the commit sequence of events
  - Supplement transaction processing
  - Allocate sequence numbers to records as they are applied to tables
  - Implement array DML



Transaction Processing Overview

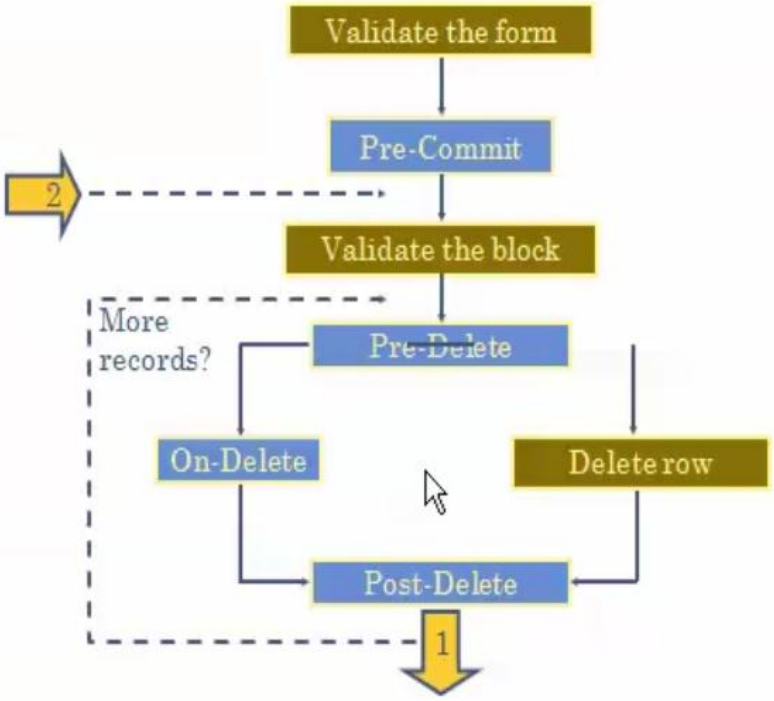
Transaction processing includes two phases:

- Post:
  - Writes record changes to base tables
  - Fires transactional triggers
- Commit: Performs database commit

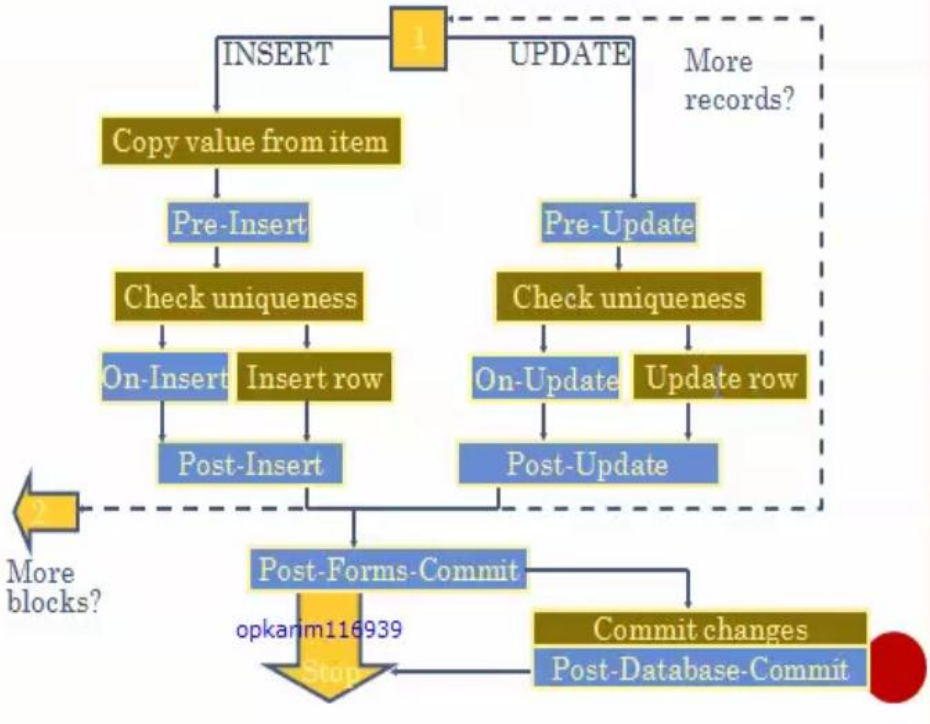
Errors result in:

- Rollback of the database changes
- Error message

The Commit Sequence of Events



The Commit Sequence of Events



Characteristics of Commit Triggers

- Pre-Commit: Fires once if form changes are made or uncommitted changes are posted
- Pre- and Post-DML
- On-DML: Fires per record, replacing default DML on row
  Use `DELETE_RECORD`, `INSERT_RECORD`, `UPDATE_RECORD` built-ins

Characteristics of Commit Triggers

- Post-Forms-Commit: Fires once even if no changes are made
- Post-Database-Commit: Fires once even if no changes are made

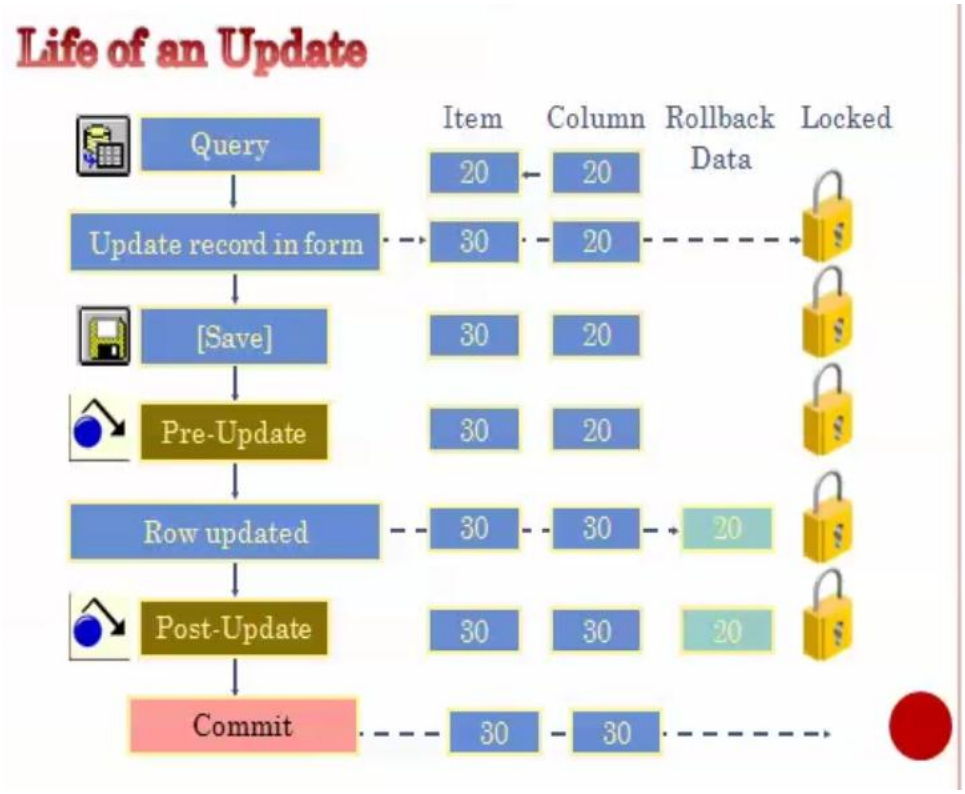
- Note: A commit-trigger failure causes a rollback to the savepoint.

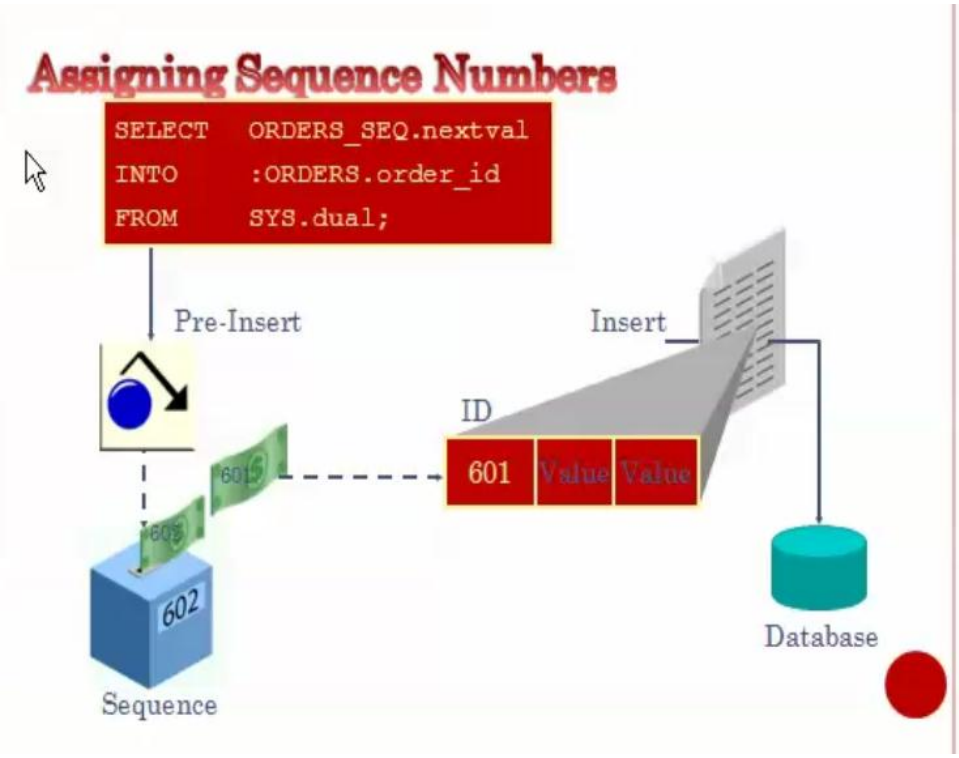
Common Uses for Commit Triggers

Pre-Commit	Check user authorization; set up special locking
Pre-Delete	Journaling; implement foreign-key delete rule
Pre-Insert	Generate sequence numbers; journaling; automatically generated columns; check constraints
Pre-Update	Journaling; implement foreign-key update rule; auto-generated columns; check constraints

Common Uses for Commit Triggers

On-Insert/Update/Delete	Replace default block DML statements
Post-Forms-Commit	Check complex multirow constraints
Post-Database-Commit	Test commit success; test uncommitted posts


Life of an Update
Item
Column
Rollback Data
Locked
Query
20
20
Update record in form
30
20
[Save]
30
20
Pre-Update
30
20
Row updated
30
30
20
Post-Update
30
30
20
Commit
30
30


Assigning Sequence Numbers
SELECT ORDERS_SEQ.nextval
INTO :ORDERS.order_id
FROM SYS.dual;
Pre-Insert
Insert
ID
601
Value
Value
602
Sequence
Database

Testing the Results of Trigger DML

- SQL%FOUND
- SQL%NOTFOUND
- SQL%ROWCOUNT

```
UPDATE ORDERS
 SET order_date = SYSDATE
 WHERE order_id = :ORDERS.order_id;
IF SQL%NOTFOUND THEN
  MESSAGE('Record not found in database');
  RAISE form_trigger_failure;
END IF;
```

DML Statements Issued During Commit Processing

```
INSERT INTO base_table   (base_column, base_column,...)
VALUES                   (:base_item, :base_item, ...)
```

```
UPDATE   base_table
SET      base_column = :base_item, base_column =
                :base_item, ...
WHERE    ROWID = :ROWID
```

```
DELETE   FROM base_table
WHERE    ROWID = :ROWID
```

Overriding Default Transaction Processing

- Additional transactional triggers:

Trigger	Do-the-Right-Thing Built-in
On-Check-Unique	CHECK_RECORD_UNIQUENESS
On-Column-Security	ENFORCE_COLUMN_SECURITY
On-Commit	COMMIT_FORM
On-Rollback	ISSUE_ROLLBACK
On-Savepoint	ISSUE_SAVEPOINT
On-Sequence-Number	GENERATE_SEQUENCE_NUMBER

Note: These triggers are meant to be used when connecting to data sources other than Oracle.

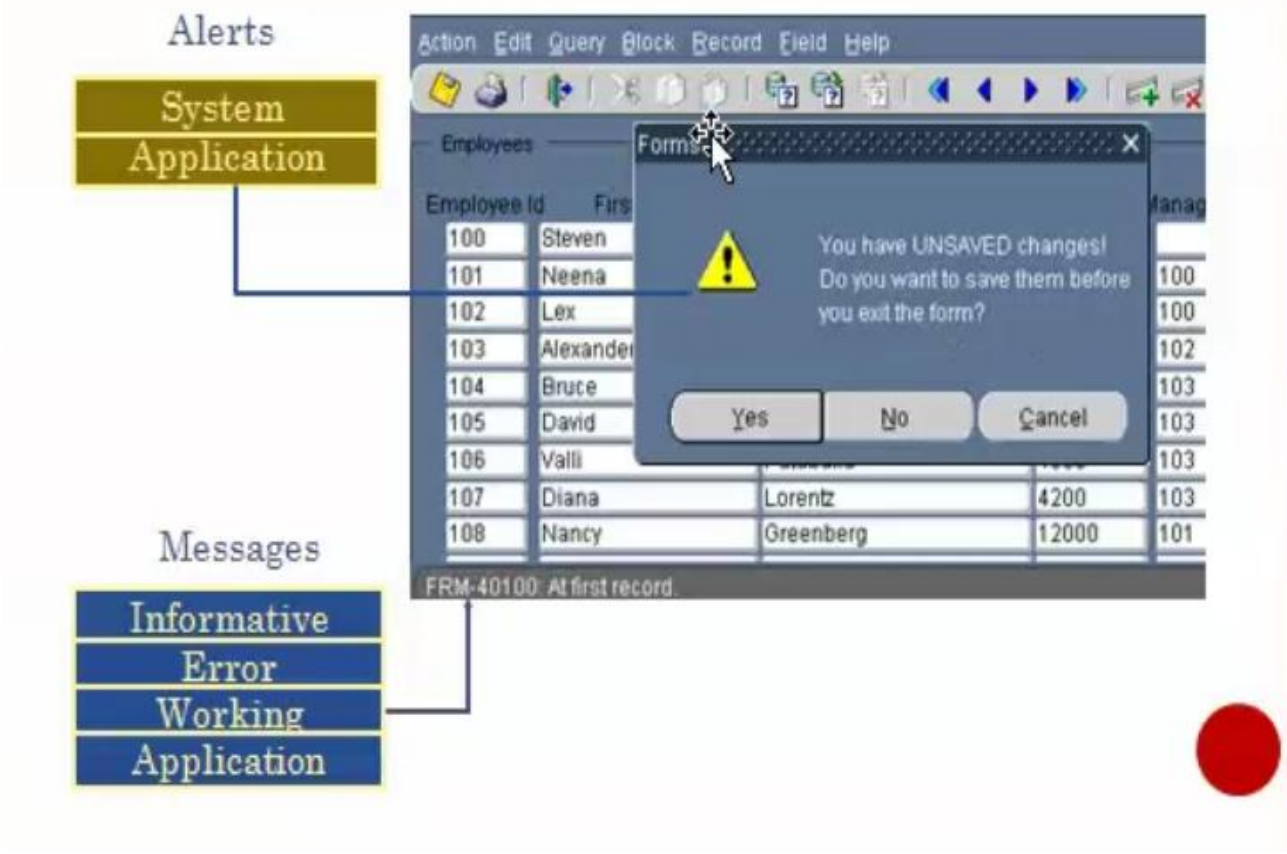
Overriding Default Transaction Processing

- Transactional triggers for logging on and off:

Trigger	Do-the-Right-Thing Built-in
Pre-Logon	-
Pre-Logout	-
On-Logon	LOGON
On-Logout	LOGOUT
Post-Logon	-
Post-Logout	-

Writing Flexible Code

Run-Time Messages and Alerts Overview



What Is Flexible Code?

- Flexible code:
  - Is reusable
  - Is generic
  - Avoids hard-coded object names
  - Makes maintenance easier
  - Increases productivity

Using System Variables for Current Context

- Input focus:
  - SYSTEM.CURSOR_BLOCK
  - SYSTEM.CURSOR_RECORD
  - SYSTEM.CURSOR_ITEM
  - SYSTEM.CURSOR_VALUE

When-Button-Pressedtrigger

```
IF :SYSTEM.CURSOR_BLOCK = 'ORDERS' THEN
  GO_BLOCK('ORDER_ITEMS');
ELSIF :SYSTEM.CURSOR_BLOCK = 'ORDER_ITEMS' THEN
  GO_BLOCK('INVENTORIES');
ELSIF :SYSTEM.CURSOR_BLOCK = 'INVENTORIES' THEN
  GO_BLOCK('ORDERS');
END IF;
```

Using System Variables for Current Context

- Trigger focus:
  - SYSTEM.TRIGGER_BLOCK
  - SYSTEM.TRIGGER_RECORD
  - SYSTEM.TRIGGER_ITEM

after the trigger fires, you may want the cursor to be in the same item instance that it was in originally

System Status Variables

- When-Button-Pressed

```
ENTER;
IF :SYSTEM.BLOCK_STATUS = 'CHANGED' THEN
  COMMIT_FORM;
END IF;
CLEAR_BLOCK;
```

Global Vairable

- No data tybe
- Used when open
- Erase when close

- Example
  - WHEN NEW FORM INSTANCE
    - :GLOBAL.NUM :='100' ;
    - :GLOBAL.NAME:='IHAB' ;
  - Post_form
    - Erase(:global.num);
    - Erase (:global.ihab);

GET_<object>_PROPERTY Built-Ins

- GET_APPLICATION_PROPERTY
- GET_FORM_PROPERTY
- GET_BLOCK_PROPERTY
- GET_RELATION_PROPERTY
- GET_RECORD_PROPERTY
- GET_ITEM_PROPERTY
- GET_ITEM_INSTANCE_PROPERTY

Example

- :GLOBAL.username := GET_APPLICATION_PROPERTY(USERNAME);
  :GLOBAL.o_sys := GET_APPLICATION_PROPERTY(OPERATING_SYSTEM);

opkarim116939

- GET_BLOCK_PROPERTY('blockname',top_record)...

- DECLARE
  cv_name varchar2(30);
  BEGIN
  cv_name := GET_ITEM_PROPERTY(:SYSTEM.CURSOR_ITEM,item_canvas);

SET_<object>_PROPERTY Built-Ins

- SET_APPLICATION_PROPERTY
- SET_FORM_PROPERTY
- SET_BLOCK_PROPERTY
- SET_RELATION_PROPERTY
- SET_RECORD_PROPERTY
- SET_ITEM_PROPERTY
- SET_ITEM_INSTANCE_PROPERTY

opkarim116939

EXAMPLE
SET_ITEM_INSTANCE_PROPERTY(:SYSTEM.CURSOR_ITEM, VISUAL_ATTRIBUTE, CURRENT_RECORD, 'VA_CURR');

SET_<object>_PROPERTY Built-Ins

- SET_LOV_PROPERTY
- SET_RADIO_BUTTON_PROPERTY
- SET_MENU_ITEM_PROPERTY
- SET_CANVAS_PROPERTY
- SET_TAB_PAGE_PROPERTY
- SET_VIEW_PROPERTY
- SET_WINDOW_PROPERTY

EXAMPLE

EXAMPLE

- SET_MENU_ITEM_PROPERTY('FILE.SAVE',ENABLED,PROPERTY_TRUE);

- DECLARE
  tbpg_id TAB_PAGE;

```
BEGIN
tbpg_id := FIND_TAB_PAGE('tab_page_1');
IF GET_TAB_PAGE_PROPERTY(tbpg_id,
enabled) = 'FALSE' THEN
 SET_TAB_PAGE_PROPERTY(tbpg_id,
enabled,property_true);
END IF;
END;
```

Referencing Objects by Internal ID

Finding the object ID:

```
lov_id := FIND_LOV('my_lov')
```



Referencing an object by ID:

```
...SHOW_LOV(lov_id)
```



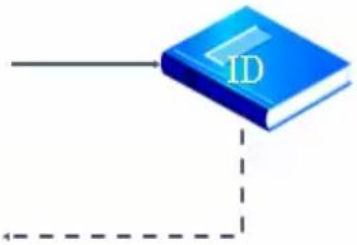
Referencing an object by name:

```
...SHOW_LOV('my_lov')
```



FIND_ Built-Ins

- FIND_ALERT
- FIND_BLOCK
- FIND_CANVAS
- FIND_EDITOR
- FIND_FORM
- FIND_ITEM
- FIND_LOV
- FIND_RELATION
- FIND_VIEW
- FIND_WINDOW



Using Object IDs

- Declare a PL/SQL variable of the same data type.
- Use the variable for any later reference to the object.
- Use the variable within the current PL/SQL block only.

Using Object IDs

- Example:

```
DECLARE
   item_var item;
BEGIN
   item_var := FIND_ITEM(:SYSTEM.CURSOR_ITEM);
   SET_ITEM_PROPERTY(item_var,position,30,55);
   SET_ITEM_PROPERTY(item_var,prompt_text,'Cur
rent');
END;
```

Referencing Objects Indirectly

- The NAME_IN function:
  - Returns:
    - The contents of variable
    - Character string
  - Use conversion functions for NUMBER and DATE

IF :CUSTOMERS.cust_last_name = 'Welles'...

IF NAME_IN('CUSTOMERS.cust_last_name') = 'Welles'...

Referencing Objects Indirectly

- The COPY procedure allows:
  - Direct copy:

```
COPY('Welles','CUSTOMERS.cust_last_name');
```

  - Indirect copy:

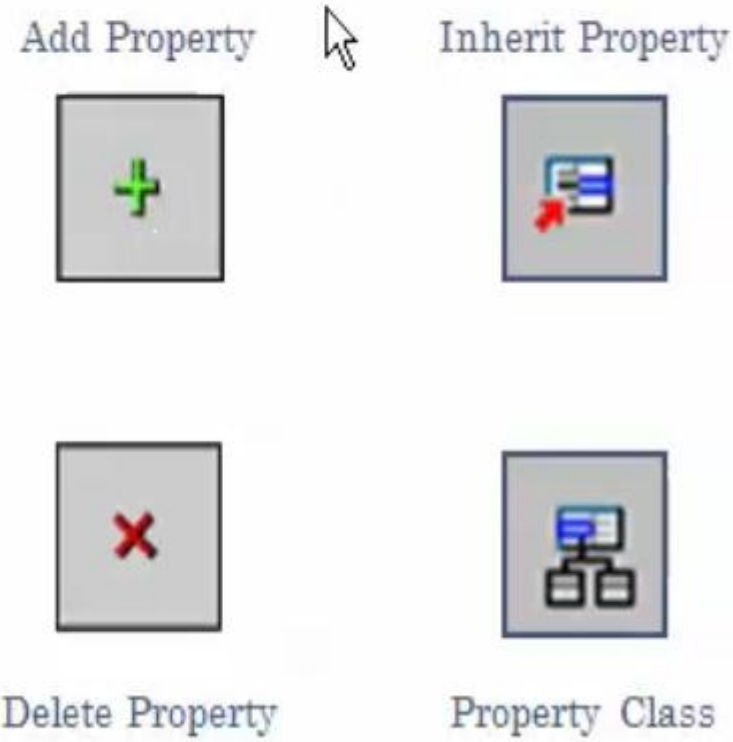
```
COPY('Welles',NAME_IN('global.customer_name_item'));
```

Sharing Objects and Code

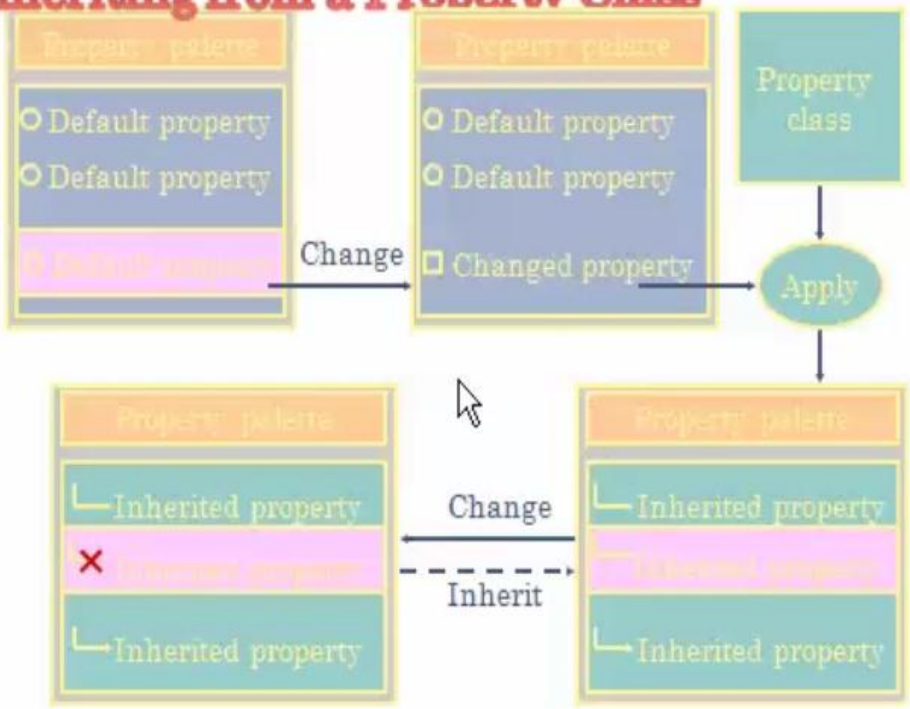
Benefits of Reusing Objects and Code

- Increases productivity
- Decreases maintenance
- Increases modularity
- Maintains standards
- Improves application performance

Creating a Property Class

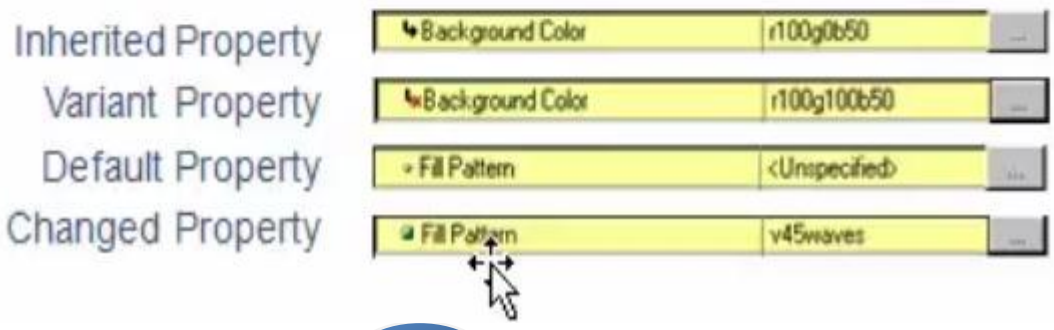


Inheriting from a Property Class



Inheriting from a Property Class

- Set the Subclass Information property.
- Convert an inherited property to a variant property.
- Convert a variant property to an inherited property.
- Convert a changed property to a default property.



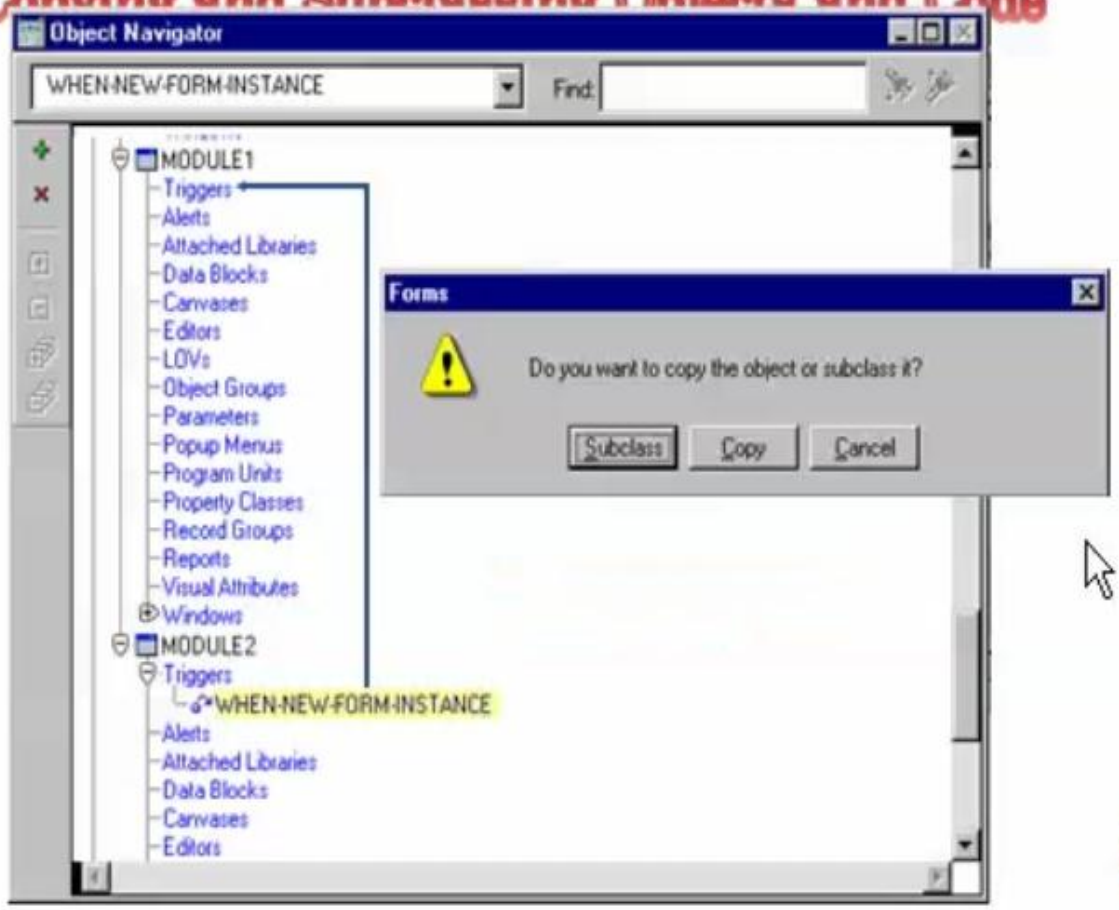
What Are Object Groups?

- Object groups:
  - Are logical containers
  - Enable you to:
    - Group related objects
    - Copy multiple objects in one operation

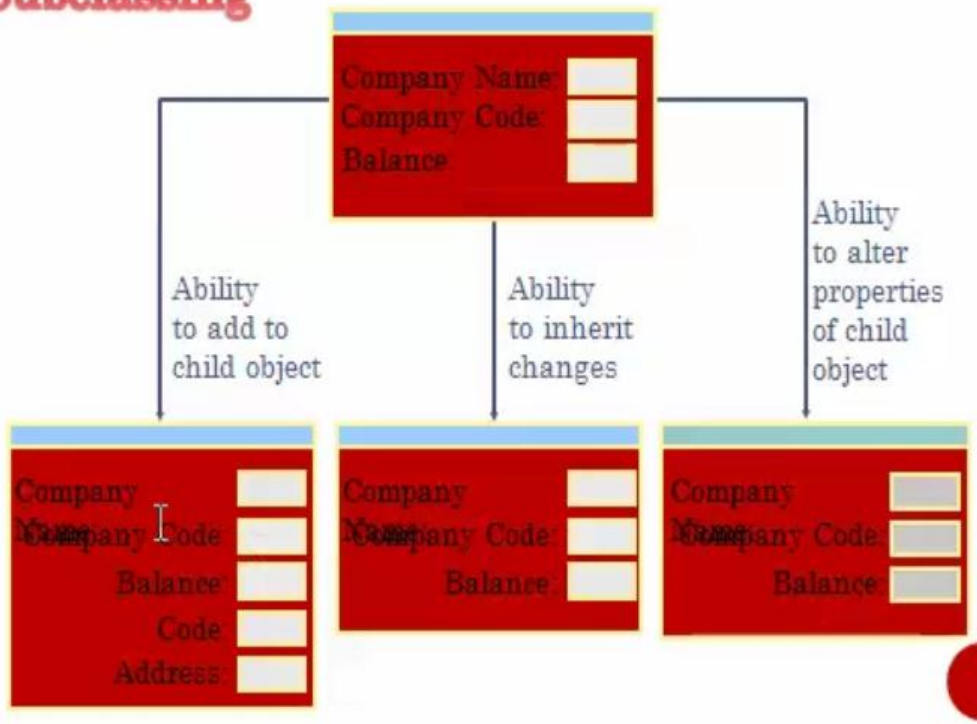
Creating and Using Object Groups

- Blocks include:
  - Items
  - Item-level triggers
  - Block-level triggers
  - Relations
- Object groups cannot include other object groups
- Deleting an object group does not affect the objects
- Deleting an object affects the object group

Copying and Subclassing Objects and Code



Subclassing



What Are Object Libraries?

- An Object Library:
  - Is a convenient container of objects for reuse
  - Simplifies reuse in complex environments
  - Supports corporate, project, and personal standards
  - Simplifies the sharing of reusable components
  - Is separate from the form module



Benefits of the Object Library

- Simplifies the sharing and reuse of objects
- Provides control and enforcement of standards
- Promotes increased network performance
- Eliminates the need to maintain multiple referenced forms



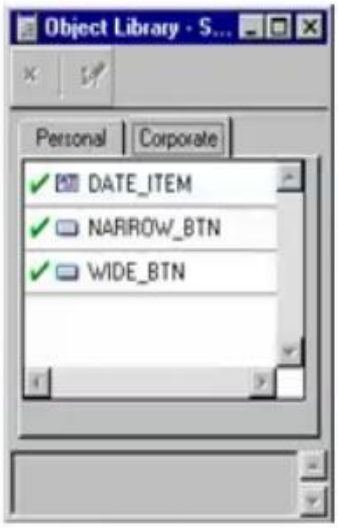
Working with Object Libraries



- Object Libraries:
  - Appear in the Navigator if they are open
  - Are used with a simple tabbed interface
  - Are populated by dragging Form objects to tab page
  - Are saved to `.olb` file

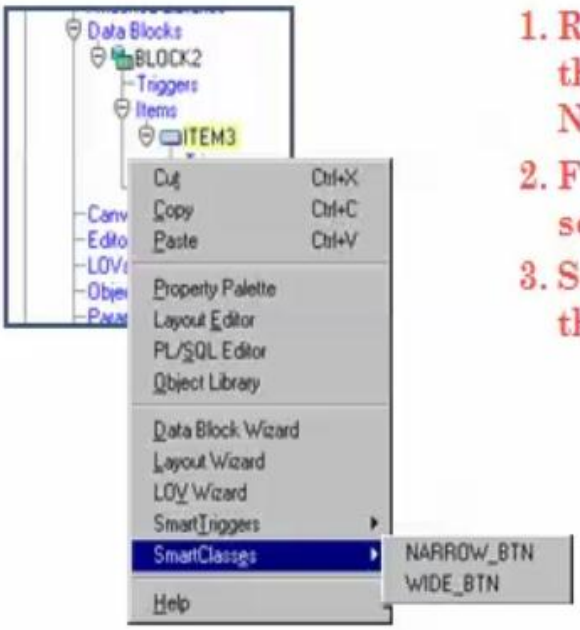
What Is a SmartClass?

- A SmartClass:
  - Is an object in an object library that is frequently used as a class
  - Can be applied easily and rapidly to existing objects
  - Can be defined in many object libraries
  - Is the preferred method to promote similarity among objects for performance
- You can have many SmartClasses of a given object type.



Check indicates a SmartClass

Working with SmartClasses



1. Right-click an object in the Layout Editor or Navigator.
2. From the pop-up menu, select SmartClasses.
3. Select a class from the list.

Reusing PL/SQL

- **Triggers:**
  - Copy and paste text
  - Copy and paste within a module
  - Copy to or subclass from another module
  - Move to an object library
- **PL/SQL program units:**
  - Copy and paste text
  - Copy and paste within a module
  - Copy to or subclass in another module
  - Create a library module
  - Move to an object library

What Are PL/SQL Libraries?

